
Aliro_Actuator

Mike Jongen

Feb 08, 2024

MODULES

1	Approach 1 (handle)	3
2	Approach 2 (command/response)	5
3	Modules	7
3.1	aliro_actuator	7
3.1.1	aliro_actuator package	7
3.1.1.1	Subpackages	7
3.1.1.2	Submodules	40
3.1.1.3	Module contents	41
4	Indices and tables	43
	Python Module Index	45
	Index	47

The code is separated into 5 parts: the 4 parts as described in the Architecture overview of the specification (chapter 5.1) and the hardware drivers.

This actuator supports both the reader side and the user side of the protocol. The class that implements the reader side is the *Reader*, and the user side is implemented by the *UserDevice*.

Both *Reader* and *UserDevice* have a similar structure. They both have a parameter `transport_protocol` (of type *TransportProtocol*) which indicates which transport protocol to use (NFC or BLE/UWB). After instantiating the class, the `transaction_initiation()` or `transaction_initiation()` method is used to set up the connection to a device. This method blocks until the connection is established.

The *UserDevice* and *Reader* can now be used in one of two ways: using the handle functions for an easy to use approach, or the command/response methods for more control.

APPROACH 1 (HANDLE)

This approach uses the attributes set during instantiation, and the ones received from commands, to make the actuator easier to use. Start by using the `start_new_session()` or `start_new_session()` to start a session. All data received will be stored in this session. The *UserDevice* now needs to wait for a command. This is done with the `wait_for_command()` method, which returns a *Command* to pass to a handle method. Now, use the methods that start with 'handle' to handle the commands:

User methods:

```
handle_select()
handle_auth0()
handle_auth1()
handle_control_flow()
handle_exchange()
handle_load_cert()
```

Reader methods:

```
handle_select()
handle_auth0()
handle_auth1()
handle_control_flow()
handle_exchange()
handle_load_cert()
```


APPROACH 2 (COMMAND/RESPONSE)

This approach gives more control to the programmer. The methods that start with ‘command’/‘response’ require a value for every piece of data they will send. The data received will also need to be manually processed by the programmer (unlike Approach 1, there is no session to save received data). Note that Approach 1 also uses these ‘command’/‘response’ methods, so you can use the Approach 1 methods as an example.

MODULES

3.1 aliro_actuator

3.1.1 aliro_actuator package

3.1.1.1 Subpackages

aliro_actuator.access_document package

Submodules

aliro_actuator.access_document.access_credential module

class aliro_actuator.access_document.access_credential.AccessDocument

Bases: *Document*

parse() → None

validate_credential() → None

class aliro_actuator.access_document.access_credential.DataElement(*id: bytes | None, schedules: list | None, access_rules: list | None, reader_rules_ids: list | None, validity_iteration: int | None*)

Bases: object

class aliro_actuator.access_document.access_credential.IssuerAuth

Bases: object

aliro_actuator.access_document.errors module

exception aliro_actuator.access_document.errors.DocumentError

Bases: Exception

exception aliro_actuator.access_document.errors.ElementSizeError(*actual: int, maximum: int | None = None, expected: int | None = None, minimum: int | None = None*)

Bases: *DocumentError*

aliro_actuator.access_document.revocation_document module

class aliro_actuator.access_document.revocation_document.**RevocationDocument**

Bases: *Document*

Module contents

class aliro_actuator.access_document.**Document**

Bases: object

retrieve() → None

store() → None

aliro_actuator.access_protocol package**Submodules****aliro_actuator.access_protocol.apdu module**

class aliro_actuator.access_protocol.apdu.**APDU**

Bases: object

Class for creating and parsing APDU messages.

Use the `parse_command/response` functions to parse the received commands/responses. Use the `create_<cmd>_command/response` to create new command/responses. The `as_bytes` attribute of these commands/responses gives a bytestring which can be send.

has the following attributes: `support_extended_length_apdu`: bool (set to true to support extended length APDU's, see 8.3 Aliro spec)

create_auth0_command(*transaction_type*: *Transaction*, *transaction_code*: *TransactionCode*,
protocol_version: int, *reader_epubk*: bytes, *transaction_identifier*: bytes,
reader_identifier: bytes, *vendor_extension*: bytes | None = None) → *Command*

create_auth0_response(*credential_epubk*: bytes, *status*: int, *cryptogram*: bytes | None = None) →
Response

create_auth1_command(*response*: *Auth1Response*, *reader_sig*: bytes, *certificate_data*: bytes | None =
None) → *Command*

create_auth1_response(*key_slot*: bytes | None, *public_key*: bytes | None, *expected_response*:
Auth1Response, *signature*: bytes, *encryption*: *EncryptionEngine*, *status*: int =
StatusBytes.SUCCESS, *private_mailbox_data*: bytes | None = None,
access_doc_retrieve: bool = False, *revocation_doc_retrieve*: bool = False,
step_up_aid_required: bool = False, *data_in_mailbox*: bool = False,
read_mailbox: bool = False, *write_mailbox*: bool = False, *send_issuer_backend*:
bool = False, *send_bound_app*: bool = False, *update_doc*: bool = False,
credential_signed_timestamp: bytes | None = None,
revocation_signed_timestamp: bytes | None = None) → *Response*

create_command(*cla: int, ins: int, p1: int, p2: int, data: bytes, le: int | None*) → *Command*

Create a command. (the other more specific functions are recommended)

create_control_flow_command(*S1: int, S2: int, domain_specific_data: bytes | None = None*) → *Command*

create_control_flow_response(*status: int*) → *Response*

create_envelope_command(*payload: bytes*) → *Command*

create_envelope_response(*payload: bytes | None = None, status: int = StatusBytes.SUCCESS*) → *Response*

create_error_response(*status_bytes: int*) → *Response*

create_exchange_command(*atomic_session: bool, payload_tlv: TLV, encryption: EncryptionEngine*) → *Command*

create_exchange_response(*payload: bytes, encryption: EncryptionEngine, status: int*) → *Response*

create_get_response_command(*payload: bytes*) → *Command*

create_get_response_response(*payload: bytes, status: int*) → *Response*

create_load_cert_command(*compressed_reader_cert: bytes*) → *Command*

create_load_cert_response(*status: int*) → *Response*

create_select_command(*aid: bytes*) → *Command*

create_select_response(*AID: bytes, type: int, expedited_phase_supported_protocol_versions: list[int], maximum_command_apdu: int | None = None, maximum_response_apdu: int | None = None, vendor_specific_tlv: TLV | None = None, status: int = StatusBytes.SUCCESS*) → *Response*

parse_command(*command_as_bytes: bytes, encryption: EncryptionEngine | None = None*) → *Command*

Parse a command bytestring. Used to extract info from a received command.

parse_response(*response_as_bytes: bytes, ins: INS, encryption: EncryptionEngine | None = None*) → *Response*

Parse a response bytestring. Used to extract info from a received response.

class `aliro_actuator.access_protocol.apdu.Auth1Response`(*value*)

Bases: `IntEnum`

Indicating the type of response requested in a auth1 command. Send with tag 0x41, bit 0. See table 8-10 of the Aliro spec.

CREDENTIAL_PUBLIC_KEY = 1

KEY_SLOT = 0

class `aliro_actuator.access_protocol.apdu.Command`

Bases: `Message`

contains an APDU command.

has the following attributes: `as_bytes: bytes` (full command as bytes) `cla: int` `ins: int` `p1: int` `p2: int` `lc: int` `data: bytes` `le: int`

classmethod **create_from_bytestring**(bytestring: bytes) → *Command*

Create a Command from a bytestring.

classmethod **create_from_parameters**(cla: int, ins: int, p1: int, p2: int, data: bytes, le: int | None) → *Command*

Create a Command from its fields.

lc is calculated from the data. Set le to None if no le field is required.

parse_as_auth0() → None

Parse this command as a AUTH0 command.

Checks the fields and raises errors for invalid fields. creates the following attributes: command_parameters: int transaction_code: int expedited_phase_protocol_version: int reader_epubk: bytes transaction_identifier: bytes reader_identifier: bytes vendor_specific_extension: bytes | None

parse_as_auth1() → None

Parse this command as a Auth1 command.

Checks the fields and raises errors for invalid fields. creates the following attributes: command_parameters: int expected_response: Auth1Response request_access_credentials: bool reader_signature: bytes certificate_data: bytes | None

parse_as_control_flow() → None

Parse this command as a Control Flow command.

Checks the fields and raises errors for invalid fields.

parse_as_envelope() → None

Parse this command as a Envelope command.

Checks the fields and raises errors for invalid fields.

parse_as_exchange(encryption: *EncryptionEngine* | None = None) → None

Parse this command as a Exchange command.

Checks the fields and raises errors for invalid fields. creates the following attributes: encrypted_payload authentication_tag decrypted_payload atomic_session payload_tlv read_requests write_requests set_requests notify ursk update_doc

parse_as_get_response() → None

Parse this command as a Get Response command.

Checks the fields and raises errors for invalid fields.

parse_as_load_cert() → None

Parse this command as a Load Cert command.

Checks the fields and raises errors for invalid fields.

parse_as_select() → None

Parse this command as a Select command.

Checks the fields and raises errors for invalid fields.

class aliro_actuator.access_protocol.apdu.**INS**(value)

Bases: IntEnum

Possible values of the INS field in an APDU message. See Table 8-2 of of the Aliro spec.

AUTH0 = 128

AUTH1 = 129

CONTROL_FLOW = 60

ENVELOPE = 195

EXCHANGE = 201

GET_RESPONSE = 192

LOAD_CERT = 209

SELECT = 164

class aliro_actuator.access_protocol.apdu.**Message**

Bases: object

Parent class of Command and Response. Contains functions that are common for both Command and Response.

to_bytes() → bytes

class aliro_actuator.access_protocol.apdu.**Response**

Bases: *Message*

contains an APDU Response.

has the following attributes: as_bytes: bytes (full command as bytes) sw1: int sw2: int status: int (combination of sw1 and sw2) data: bytes

classmethod **create_from_bytestring**(bytestring: bytes) → *Response*

Create a Response from a bytestring.

classmethod **create_from_parameters**(data: bytes | None = None, status: int = StatusBytes.SUCCESS) → *Response*

Create a Command from its fields.

parse_as_auth0() → None

Parse this response as a Auth0 response.

creates the following attributes: credential_epubk: bytes cryptogram: bytes (if present) vendor_specific_extensions: tlv (if present)

parse_as_auth1(encryption: *EncryptionEngine* | None = None) → None

Parse this response as a Auth1 response.

creates the following attributes: encrypted_payload: bytes authentication_tag: bytes decrypted_payload: bytes payload_tlv: TLV key_slot: bytes | None credential_public_key: bytes | None user_device_signature: bytes private_mailbox_data: bytes | None signaling_bitmap: bytes credential_signed_timestamp: bytes | None revocation_signed_timestamp: bytes | None

parse_as_control_flow() → None

Parse this response as a control_flow response.

parse_as_envelope() → None

Parse this response as a envelope response.

parse_as_exchange(*encryption*: [EncryptionEngine](#) | *None* = *None*) → *None*

Parse this response as a Exchange response.

creates the following attributes: *encrypted_payload*: bytes *authentication_tag*: bytes *decrypted_payload*: bytes *status_code*: bytes *read_data*: bytes

parse_as_get_response() → *None*

Parse this response as a get_response response.

parse_as_load_cert() → *None*

Parse this response as a Auth1 response.

parse_as_select() → *None*

Parse this response as a Select response.

creates the following attributes: *compl_aid*: bytes *proprietary_tlv*: TLV type: bytes *expedited_phase_supported_protocol_versions*: bytes *maximum_command_apdu*: int *maximum_response_apdu*: int *vendor_specific_extensions*: TLV

property sw1: int

property sw2: int

class `aliro_actuator.access_protocol.apdu.S1`(*value*)

Bases: `IntEnum`

Indicating the S1 parameter in a control flow command. Send with tag 0x41. See table 8-13 and 8.5.7.2 of the Aliro spec.

FINISHED_WITH_FAILURE = 0

FINISHED_WITH_SUCCESS = 1

class `aliro_actuator.access_protocol.apdu.S2`(*value*)

Bases: `IntEnum`

Indicating the S2 parameter in a control flow command. Send with tag 0x42. See table 8-13 and 8.5.7.3 of the Aliro spec.

NONE = 0

PROTOCOL_VERSION_NOT_SUPPORTED = 39

class `aliro_actuator.access_protocol.apdu.StatusBytes`(*value*)

Bases: `IntEnum`

Indicating (some) known values of the status bytes returned in a response.

COMMAND_NOT_ALLOWED = 26880

COMMAND_NOT_COMPLIANT = 26369

CONDITIONS_OF_USE_NOT_SATISFIED = 27013

FILE_OR_APP_NOT_FOUND = 27266

FUNCTIONS_IN_CLA_NOT_SUPPORTED = 26624

FUNCTION_NOT_SUPPORTED = 27265

```

GENERIC_ERROR = 25600
INCORRECT_P1_P2 = 27392
INCORRECT_PARAMETERS_IN_DATA = 27264
INCORRECT_SECURE_MESSAGING_DOS = 27016
INVALID_CLASS = 28160
INVALID_INSTRUCTION = 27904
LOGICAL_CHANNEL_NOT_SUPPORTED = 26753
MEMORY_FAILURE = 25985
NO_PRECISE_DIAGNOSIS = 28160
SECURITY_STATUS_NOT_SATISFIED = 27010
SUCCESS = 36864
WRONG_LENGTH_IN_LC = 26368

```

```
class aliro_actuator.access_protocol.apdu.Transaction(value)
```

Bases: IntEnum

Indicating the transaction type in a auth0 command. See table 8-5 of the Aliro spec.

```
FAST = 1
```

```
STANDARD = 0
```

```
class aliro_actuator.access_protocol.apdu.TransactionCode(value)
```

Bases: IntEnum

Indicating the transaction code in a auth0 command. See table 8-1 and 8-3 of the Aliro spec.

```
FORCE_USER_AUTHENTICATION = 3
```

```
USER_DEVICE = 1
```

```
USER_DEVICE_SECURE_ACTION = 2
```

aliro_actuator.access_protocol.authentication module

```
aliro_actuator.access_protocol.authentication.create_reader_authentication(reader_identifier:
    bytes,
    credential_epubk:
    PublicKey,
    reader_epubk:
    PublicKey, trans-
    action_identifier:
    bytes) → TLV
```

```
aliro_actuator.access_protocol.authentication.create_user_device_authentication(reader_identifier:  
                                     bytes,  
                                     credential_epubk:  
                                     PublicKey,  
                                     reader_epubk:  
                                     PublicKey,  
                                     transaction_identifier:  
                                     bytes) →  
                                     TLV
```

aliro_actuator.access_protocol.defines module

```
class aliro_actuator.access_protocol.defines.Auth0
```

```
    Bases: object
```

```
    COMMAND_LEN = 1
```

```
    COMMAND_TAG = 65
```

```
    CREDENTIAL_EPUBK_LEN = 65
```

```
    CREDENTIAL_EPUBK_TAG = 134
```

```
    CRYPTOGRAM_LEN = 64
```

```
    CRYPTOGRAM_TAG = 157
```

```
    ETPV_LEN = 2
```

```
    ETPV_TAG = 92
```

```
    READER_EPUBK_LEN = 65
```

```
    READER_EPUBK_TAG = 135
```

```
    READER_IDENTIFIER_LEN = 32
```

```
    READER_IDENTIFIER_TAG = 77
```

```
    RE_VENDOR_SPECIFIC_MAX_LEN = 128
```

```
    RE_VENDOR_SPECIFIC_TAG = 178
```

```
    TRANSACTION_CODE_LEN = 1
```

```
    TRANSACTION_CODE_TAG = 66
```

```
    TRANSACTION_ID_LEN = 16
```

```
    TRANSACTION_ID_TAG = 76
```

```
    VENDOR_SPECIFIC_MAX_LEN = 128
```

```
    VENDOR_SPECIFIC_TAG = 177
```

```
class aliro_actuator.access_protocol.defines.Auth1
    Bases: object
    CERTIFICATE_TAG = 144
    COMMAND_LEN = 1
    COMMAND_TAG = 65
    CREDENTIAL_PUBK_LEN = 65
    CREDENTIAL_PUBK_TAG = 90
    CREDENTIAL_TIMESTAMP_LEN = 20
    CREDENTIAL_TIMESTAMP_TAG = 145
    KEY_SLOT_LEN = 8
    KEY_SLOT_TAG = 78
    MAILBOX_DATA_TAG = 75
    READER_SIG_LEN = 64
    READER_SIG_TAG = 158
    REVOCATION_TIMESTAMP_LEN = 20
    REVOCATION_TIMESTAMP_TAG = 146
    SIGNALING_BITMAP_LEN = 2
    SIGNALING_BITMAP_TAG = 94
    USER_DEVICE_SIG_LEN = 64
    USER_DEVICE_SIG_TAG = 158

class aliro_actuator.access_protocol.defines.ControlFlow
    Bases: object
    DOMAIN_SPECIFIC_TAG = 67
    S1_TAG = 65
    S2_TAG = 66

class aliro_actuator.access_protocol.defines.Exchange
    Bases: object
    NOTIFY_TAG = 174
    READ_TAG = 135
    SET_TAG = 149
    UPDATE_DOC_TAG = 129
    URSK_TAG = 152
```

```
WRITE_TAG = 138

class aliro_actuator.access_protocol.defines.ReaderAuth
    Bases: object
    CREDENTIAL_EPUBK_TAG = 134
    READER_EPUBK_TAG = 135
    READER_IDENTIFIER_TAG = 77
    TRANSACTION_IDENTIFIER_TAG = 76
    USAGE = b'A]\x95i'
    USAGE_TAG = 147

class aliro_actuator.access_protocol.defines.Select
    Bases: object
    AID_LEN = 9
    AID_TAG = 132
    ETSPV_TAG = 92
    EXTENDED_INFO_LEN = 8
    EXTENDED_INFO_TAG = 32614
    FCI_TAG = 111
    MAX_COMMAND_TAG = 2
    MAX_RESPONSE_TAG = 2
    PROPRIETARY_TAG = 165
    TYPE_LEN = 2
    TYPE_TAG = 128
    VENDOR_SPECIFIC_TAG = 179

class aliro_actuator.access_protocol.defines.TransportProtocol(value)
    Bases: IntEnum
    An enumeration.
    BLE_UWB = 1
    NFC = 0
    SOCKET_BLE = 3
    SOCKET_NFC = 2

class aliro_actuator.access_protocol.defines.UserDeviceAuth
    Bases: object
    CREDENTIAL_EPUBK_TAG = 134
```

```

READER_EPUBK_TAG = 135

READER_IDENTIFIER_TAG = 77

TRANSACTION_IDENTIFIER_TAG = 76

USAGE = b'N\x88{L'

USAGE_TAG = 147

```

aliro_actuator.access_protocol.encryption module

class aliro_actuator.access_protocol.encryption.**DeviceType**(*value*)

Bases: Enum

Enumerator, used by the EncryptionEngine, to indicate if the device is a reader or user.

READER = 1

USER = 2

class aliro_actuator.access_protocol.encryption.**EncryptionEngine**(*devicetype: DeviceType,*
exchange_SK_reader: bytes,
exchange_SK_device: bytes)

Bases: object

class to encrypt and decrypt messages. Uses AES-256.

Args:

devicetype (DeviceType): indicates if the device creating this class is a user or reader
exchange_SK_reader (bytes): exchange SK Reader as described in Figure 8-13 of the Aliro spec
exchange_SK_device (bytes): exchange SK Device as described in Figure 8-13 of the Aliro spec

check_counters_valid() → bool

Checks the counters as required for the exchange command (Figure 8-15 of the Aliro spec)

Returns:

bool: True if the counters are valid (< 0xFFFF), else False

decrypt(*ciphertext: bytes, authentication_tag: bytes, ad: bytes = b''*) → bytes

decrypts encrypted data and indicates if verification succeeded

(figure 8-10 of the Aliro spec)

Args:

ciphertext (bytes): Ciphertext. Can be any size. authentication_tag (bytes): Authentication Tag. Must be 16 bytes. ad (bytes, optional): Additional data. Defaults to b''.

Raises:

VerificationError: raised when the verification fails.

Returns:

bytes: decrypted plaintext

encrypt(*data: bytes, ad: bytes = b''*) → tuple[bytes, bytes]

Generates encrypted data + authentication tag

(figure 8-9 of the Aliro spec)

Args:

data (bytes): Data to encrypt. Can be any size. ad (bytes, optional): Additional data. Defaults to b"".

Returns:

tuple[bytes, bytes]: tuple of the encrypted data and the authentication tag.

exception `aliro_actuator.access_protocol.encryption.VerificationError`

Bases: `AccessProtocolError`

Raised when the verification of an AES decryption fails.

`aliro_actuator.access_protocol.encryption.create_proprietary_information`(*type: int, expedited_phase_supported_protocol_versions: list[int], maximum_command_apdu: int | None = None, maximum_response_apdu: int | None = None, vendor_specific_tlv: TLV | None = None*) $\rightarrow$ *TLV*

Creates the proprietary information (see Table 10-2), used for salt generation and Select command.

Args:

type (int): Application type (see Table 10-3) expedited_phase_supported_protocol_versions (list[int]): List of supported protocol versions. maximum_command_apdu (int | None, optional): Defaults to None. maximum_response_apdu (int | None, optional): Defaults to None. vendor_specific_tlv (TLV | None, optional): Defaults to None.

Returns:

TLV: Proprietary information (as TLV)

`aliro_actuator.access_protocol.encryption.create_salt`(*transport_protocol: TransportProtocol, word: bytes, reader_public_key: PublicKey, reader_ephemeral_public_key: PublicKey, reader_identifier: bytes, protocol_version: bytes, transaction_identifier: bytes, flag: bytes, proprietary_information: bytes, credential_ephemeral_public_key: PublicKey | None = None*) $\rightarrow$ bytes

Generates the salt used for key generation

Args:

transport_protocol (TransportProtocol): The transport protocol used. word (bytes): "VolatileFast", "Volatile*****" or "Persistent***". reader_public_key (PublicKey). reader_ephemeral_public_key (PublicKey). reader_identifier (bytes). protocol_version (bytes). transaction_identifier (bytes). flag (bytes): command_parameters || transaction_code. proprietary_information (bytes): proprietary information. credential_ephemeral_public_key (PublicKey | None, optional): only for "VolatileFast" or "Persistent***". Defaults to None.

Returns:

bytes: the salt as bytes

aliro_actuator.access_protocol.errors module**exception** `aliro_actuator.access_protocol.errors.AccessProtocolError`Bases: `Exception`

Parent class of all errors related to the access protocol.

exception `aliro_actuator.access_protocol.errors.CreateCommandError`Bases: `AccessProtocolError`**exception** `aliro_actuator.access_protocol.errors.CreateResponseError`Bases: `AccessProtocolError`**exception** `aliro_actuator.access_protocol.errors.InvalidAIDError`(*command: bytes, aid: bytes*)Bases: `InvalidCommandDataError`

Raised when a command with an invalid AID is received.

exception `aliro_actuator.access_protocol.errors.InvalidCLAError`(*command: bytes*)Bases: `InvalidCommandError`

Raised when the received command has an invalid CLA.

exception `aliro_actuator.access_protocol.errors.InvalidCommandDataError`(*command: bytes, message: str | None = None*)Bases: `InvalidCommandError`

Raised when the received command has an invalid Data field.

exception `aliro_actuator.access_protocol.errors.InvalidCommandError`(*command: bytes*)Bases: `AccessProtocolError`

Raised when the received command is invalid.

exception `aliro_actuator.access_protocol.errors.InvalidINSError`(*command: bytes*)Bases: `InvalidCommandError`

Raised when the received command has an invalid INS.

exception `aliro_actuator.access_protocol.errors.InvalidLcError`(*command: bytes*)Bases: `InvalidCommandError`

Raised when the received command has an invalid LC.

exception `aliro_actuator.access_protocol.errors.InvalidLeError`(*command: bytes*)Bases: `InvalidCommandError`

Raised when the received command has an invalid Le.

exception `aliro_actuator.access_protocol.errors.InvalidParameterError`(*command: bytes*)Bases: `InvalidCommandError`

Raised when the received command has an invalid Parameter (P1 or P2).

exception `aliro_actuator.access_protocol.errors.InvalidResponseDataError`(*response: bytes, other_info: str = ""*)Bases: `InvalidResponseError`

Raised when the received Response has an invalid data.

exception `aliro_actuator.access_protocol.errors.InvalidResponseError(response: bytes)`

Bases: `AccessProtocolError`

Raised when the received response is invalid.

exception `aliro_actuator.access_protocol.errors.InvalidStatusError(response: bytes, status: int)`

Bases: `InvalidResponseError`

Raised when the received Response has an invalid status.

exception `aliro_actuator.access_protocol.errors.MessageTooLongError`

Bases: `AccessProtocolError`

Raised when the received command is too long. See 8.3 of the Aliro Spec.

exception `aliro_actuator.access_protocol.errors.SessionError`

Bases: `AccessProtocolError`

exception `aliro_actuator.access_protocol.errors.UnexpectedCommandError`

Bases: `AccessProtocolError`

Raised when a different command is received than expected.

exception `aliro_actuator.access_protocol.errors.UnexpectedResponseError`

Bases: `AccessProtocolError`

Raised when a different Response is received than expected.

exception `aliro_actuator.access_protocol.errors.VersionError`

Bases: `AccessProtocolError`

Raised when the requested version is not supported.

`aliro_actuator.access_protocol.mailbox` module

class `aliro_actuator.access_protocol.mailbox.Mailbox(initial_data: list[tuple[bytes, int, bytes]] | None = None, size: int = 0, read_permission: bool = True, write_permission: bool = True)`

Bases: `object`

DATA_TAG = 130

INDEX_TAG = 129

START_OF_DATA_TAG = 96

check_boundaries(*offset: int, length: int*) → bool

Checks if the offset and length combination is a valid option for this mailbox.

Args:

offset (int): Offset in bytes. *length* (int): Data to write.

Returns:

bool: True if offset and length fit in this mailbox, else False

data_is_set() → bool

Checks if some data is different from zeros

Returns:

bool: True if not all data is 0x00, else False

get_data() → bytes

Returns just the mailbox data (no index, tags or lengths).

Returns:

bytes: mailbox data, not including tag (0x82) and length.

get_raw() → bytes

Returns the mailbox contents as bytes.

Returns:

bytes: mailbox contents (including index and data)

get_tlv() → *TLV*

Returns the mailbox contents as a TLV.

Returns:

TLV: Mailbox contents.

read(offset: int, length: int) → bytes

Returns a block of data from the mailbox.

Args:

offset (int): offset in bytes. length (int): length in bytes.

Raises:

MailboxPermissionError: raised if read_permission is not set.

Returns:

bytes: Data from offset to offset + length

set(offset: int, length: int, value: int) → None

Set a block of data to a single value.

Args:

offset (int): offset in bytes. length (int): length in bytes value (int): Every byte from offset to offset + length is set to this value.

Raises:

MailboxPermissionError: raised if write_permission is not set.

write(offset: int, data: bytes) → None

Write a block of data to the mailbox.

Args:

offset (int): Offset in bytes. data (bytes): Data to write.

Raises:

MailboxPermissionError: raised if write_permission is not set.

exception aliro_actuator.access_protocol.mailbox.**MailboxFormatError**

Bases: *AccessProtocolError*

Raised when data passed to the mailbox has the wrong format.

exception aliro_actuator.access_protocol.mailbox.**MailboxPermissionError**

Bases: *AccessProtocolError*

Raised when trying to read/write the mailbox without correct permissions.

aliro_actuator.access_protocol.reader module

```
class aliro_actuator.access_protocol.reader.Reader(transport_protocol: TransportProtocol,
                                                    transport_override: TransportProtocolBase |
                                                    None = None, reader_group_identifier: bytes |
                                                    None = None, reader_group_sub_identifier: bytes
                                                    | None = None, reader_cert: bytes | None = None,
                                                    reader_key: KeyPair | None = None,
                                                    vendor_extension: bytes | None = None)
```

Bases: *Device*

Simulates a reader device.

Args:

transport_protocol (TransportProtocol): Transport protocol to use. transport_override (TransportProtocolBase | None, optional): Override the transport protocol. Mainly used for testing. Defaults to None. reader_group_identifier (bytes | None, optional): Part of the reader_identifier. Defaults to None. reader_group_sub_identifier (bytes | None, optional): Part of the reader_identifier. Defaults to None. reader_cert (bytes | None, optional): Reader certificate. Defaults to None. reader_key (KeyPair | None, optional): Reader Key. Defaults to None.

Raises:

AccessProtocolError: Raised when arguments have invalid format.

```
command_auth0(transaction: Transaction, transaction_code: TransactionCode, protocol_version: int,
               reader_epubk: bytes, transaction_identifier: bytes, reader_identifier: bytes,
               vendor_extension: bytes | None = None) → Response
```

Create and send a auth0 command.

Args:

transaction (Transaction): fast or standard transaction_code (TransactionCode): code with instruction (ex. Lock/Unlock) protocol_version (int): reader_epubk (bytes): Reader Ephemeral Key as bytes transaction_identifier (bytes): reader_identifier (bytes): vendor_extension (bytes | None): Vendor specific extension TLV. Defaults to None.

Returns:

Response: Response containing the received data.

```
command_auth1(expected_response: Auth1Response, reader_identifier: bytes, credential_epubk: PublicKey,
               reader_epubk: PublicKey, transaction_identifier: bytes, encryption: EncryptionEngine |
               None = None) → Response
```

Create and send a auth1 command.

Args:

expected_response (Auth1Response): key slot or credential public key reader_identifier (bytes): credential_epubk (PublicKey): reader_epubk (PublicKey): transaction_identifier (bytes): encryption (EncryptionEngine | None, optional): Encryption engine to decrypt the response. Response will not be decrypted if this is None. Defaults to None.

Returns:

Response: Response containing the received data.

```
command_control_flow(s1: int, s2: int, domain_specific_data: bytes | None = None) → Response
```

Create and send a exchange command.

Args:

s1 (int): s2 (int): domain_specific_data (bytes | None, optional): Defaults to None.

Returns:

Response: Response containing the received data.

command_envelope() → None

command_exchange(*atomic_session*: bool, *payload*: TLV, *encryption*: EncryptionEngine) → Response

Create and send a exchange command.

Args:

atomic_session (bool): if True, this is part of an atomic session payload (TLV): The payload to send.
encryption (EncryptionEngine): Encryption engine to encrypt the message and decode the response.

Returns:

Response: Response containing the received data.

command_get_response() → None

command_load_cert(*compressed_cert*: bytes) → Response

Create and send a load_cert command.

Args:

compressed_cert (bytes): compressed certificate to send.

Returns:

Response: Response containing the received data.

command_select(*aid*: bytes) → Response

Create and send a select command.

Args:

aid (bytes): AID to be send.

Returns:

Response: Response containing the received data.

expedited_transaction_fast() → None

expedited_transaction_standard(*transaction_code*: TransactionCode, *load_cert*: bool = False) → None

Runs the Expedited Standard Phase.

Args:

transaction_code (TransactionCode): Passed during AUTH0. *load_cert* (bool, optional): Runs the load_cert command if True. Defaults to False.

failure_process() → None

Should be called when a failure state has occurred. Destroys all session bound keys and data. If transport protocol is NFC, a control_flow command indicating failure is send. If transport protocol is BLE, a failure event message is send.

handle_auth0(*transaction_type*: Transaction, *transaction_code*: TransactionCode) → None

Create and send a AUTH0 command. Required data from is retrieved from the Reader (self) and the session. The data contained in the response is stored in the session.

Args:

transaction_type (Transaction): fast or standard *transaction_code* (TransactionCode): code with instruction (ex. Lock/Unlock)

Raises:

SessionError: Raised if no session is found. AccessProtocolError: Raised if the response has invalid data.

handle_auth1(*expected_response*: `Auth1Response = Auth1Response.CREDENTIAL_PUBLIC_KEY`) → None

Create and send a AUTH1 command. Required data from is retrieved from the Reader (self) and the session. The data contained in the response is stored in the session.

Raises:

SessionError: Raised if no session is found.

handle_control_flow(*success*: `bool`) → None

Create and send a control_flow command. Required data from is retrieved from the Reader (self) and the session. The data contained in the response is stored in the session.

Args:

success (bool): if True, the command will indicate a success status.

Raises:

SessionError: Raised if no session is found.

handle_exchange(*atomic_session*: `bool`, *read_requests*: `list[tuple[int, int]] | None`, *write_requests*: `list[tuple[int, bytes]] | None`, *set_requests*: `list[tuple[int, int, int]] | None`, *notify*: `TLV | None = None`, *ursk*: `bytes | None = None`, *update_doc*: `bytes | None = None`) → `list[bytes]`

Create and send a exchange command. Required data from is retrieved from the Reader (self) and the session. The data contained in the response is stored in the session.

Args:

atomic_session (bool): if True, this is part of an atomic session read_requests (list[tuple[int, int]] | None): List of (offset, length) tuples. write_requests (list[tuple[int, bytes]] | None): list of (offset, data) tuples. set_requests (list[tuple[int, int, int]] | None): list of (offset, length, value) tuples. notify (TLV | None): Notify TLV ursk (bytes | None): URSK, for BLE update_doc (bytes | None): request update of an existing Access Document

Raises:

SessionError: Raised if no session is found.

Returns:

list[bytes]: list of read data.

handle_load_cert() → None

Create and send a load_cert command. Required data from is retrieved from the Reader (self) and the session. The data contained in the response is stored in the session.

Raises:

SessionError: Raised if no session is found. AccessProtocolError: Raised if the response has invalid data.

handle_select(*aid*: `bytes`) → None

create and send a select command. Required data from is retrieved from the Reader (self) and the session. The data contained in the response is stored in the session.

Args:

aid (bytes): AID to be send.

Raises:

SessionError: Raised if no session is found. AccessProtocolError: Raised if the response has invalid data. UnexpectedResponseError: Raised if the response has status/data that cannot be handled

property reader_group_identifier: `bytes`

property reader_group_sub_identifier: `bytes`

property reader_identifier: bytes

start_new_session(*transaction_identifier: bytes | None = None, ephemeral_key: KeyPair | None = None*)
→ None

Start a new reader session. Must be done before using handle commands. This sessions stores all information received from commands. Start a new session to delete all received info and start over.

Args:

transaction_identifier (bytes | None, optional): Transaction identifier used for this session. Randomly generated if None. Defaults to None. *ephemeral_key* (KeyPair | None, optional): ephemeral reader key used for the session. Randomly generated if None. Defaults to None.

step_up_transaction() → None

transaction_initiation() → None

Initializes the hardware and sets up a connection to the card.

class aliro_actuator.access_protocol.reader.**ReaderSession**(*reader_key: KeyPair, reader_identifier: bytes, vendor_extension: bytes | None = None*)

Bases: object

Contains info from a single session (with one User Device)

can_retrieve_access_credential() → bool

can_retrieve_revocation_document() → bool

check_user_device_authentication(*user_device_signature: bytes*) → bool

decrypt_payload(*encrypted_payload: bytes, authentication_tag: bytes*) → bytes

derive_key_volatile(*transport_protocol: TransportProtocol*) → None

encrypt_payload(*payload: bytes*) → tuple[bytes, bytes]

generate_ephemeral_key(*ephemeral_key: KeyPair | None = None*) → None

get_credential_ephemeral_key() → bytes

get_reader_epubkey() → *PublicKey*

lookup_credential_public_key(*key_slot: bytes*) → *PublicKey*

property reader_group_identifier: bytes

property reader_group_sub_identifier: bytes

property reader_identifier: bytes

set_auth1_info(*auth1_response: Response*) → None

set_credential_ephemeral_key(*key: PublicKey*) → None

set_credential_public_key(*key: PublicKey*) → None

set_flag(*transaction: Transaction, transaction_code: TransactionCode*) → None

set_response_vendor_extension(*vendor_extension: bytes | None*) → None

set_select_info(*select_response*: [Response](#)) → None

set_shared_key() → None

step_up_aid_select_required() → bool

property transaction_identifier: bytes

aliro_actuator.access_protocol.tlv module

class `aliro_actuator.access_protocol.tlv.TLV`(*data*: list[tuple[int, bytes | list]])

Bases: object

Class to handle TLV structures (BER-TLV, ISO 7816-4) (use the `from_bytes` method to create this struct from a bytestring)

Args:

data (list[tuple[int, bytes | list]]): list of tuples. The tuples consist of tags and values. The values can be either bytes or a list of tag/value tuples. (length is calculated using the data and does not need to be specified)

add_value(*tag*: int, *data*: bytes) → None

add a new tag/value pair to the TLV

Args:

tag (int): tag data (bytes): value

static from_bytes(*data_bytes*: bytes) → [TLV](#)

Converts tlv value (BER-TLV, ISO 7816-4) from bytestring to TLV class Value is b"" if the tlv tag has no value. (length is not represented in the class, but can be derived from the value)

Raises:

TlvError: error during parsing of the bytestring

get_all_bytes_of_tag(*tag*: int) → list[bytes]

Similar to `get_all_of_tag`, but the list only contains bytes objects. Raises an error when a no byte object is found.

Args:

tag (int): tag number

Raises:

TlvError: Raised when value is not of type bytes.

Returns:

list[bytes]: list with all values with given tag

get_all_of_tag(*tag*: int) → list[bytes | [TLV](#)]

returns a list with all values with a given tag.

Args:

tag (int): tag number

Returns:

list[bytes | TLV]: list with all values with given tag. Can be empty if tag was not found.

get_bytes(tag: int, index: int = 0) → bytes

Similar to get_value, but always returns a bytestring. Raises error when requested element is not a bytestring

Args:

tag (int): tag number index (int, optional): when there are multiple elements with the same tag, use this to differentiate them. Defaults to 0.

Raises:

IndexError: Raised when a tag is not found. TlvError: Raised when value is not of type bytes.

Returns:

bytes: value

get_tlv(tag: int, index: int = 0) → TLV

Similar to get_value, but always returns a TLV object. Raises error when requested element is not a TLV object

Args:

tag (int): tag number index (int, optional): when there are multiple elements with the same tag, use this to differentiate them. Defaults to 0.

Raises:

IndexError: Raised when a tag is not found. TlvError: Raised when value is not of type TLV.

Returns:

TLV: value

get_value(tag: int, index: int = 0) → bytes | TLV

Get the value of a tag. use index if there are multiple tags with the same name

Args:

tag (int): tag to find. index (int, optional): use to differentiate multiple values with the same tag. Defaults to 0.

Raises:

IndexError: Raised when a tag is not found.

Returns:

bytes | TLV: the value as bytes or TLV. Type depends on the structure of the TLV.

to_bytes() → bytes

Converts tlv value (BER-TLV, ISO 7816-4) to bytestring. (length is derived from the value)

Raises:

TlvError: error during construction of the bytestring

to_data() → list

returns the tlv as a list of tuples

exception aliro_actuator.access_protocol.tlv.TlvError

Bases: [AccessProtocolError](#)

Raised when a TLV is malformed, has unexpected items or is missing items.

aliro_actuator.access_protocol.user_device module**class** aliro_actuator.access_protocol.user_device.**MailboxSession**

Bases: object

class **Command**(value)

Bases: Enum

An enumeration.

SET = 0**WRITE** = 1**add_set**(data: bytes | None) → None**add_write**(data: bytes | None) → None**execute_commands**(mailbox: Mailbox) → None**is_started**() → bool**start**() → None**stop**() → None**class** aliro_actuator.access_protocol.user_device.**UserDevice**(transport_protocol: TransportProtocol, transport_override: TransportProtocolBase | None = None, access_credentials: list[AccessCredential] = [], supported_versions: list[int] = [256], mailbox: int | list[tuple[bytes, int, bytes]] | None = None, mailbox_read: bool = True, mailbox_write: bool = True, vendor_extension: bytes | None = None, fast_transaction_implemented: bool = True)

Bases: Device

Simulates a user device.

Args:

transport_protocol (TransportProtocol): The transport protocol to use. access_credentials (list[bytes], optional): list of access_credentials. Defaults to []. supported_versions (list[int], optional): List of supported protocol versions. Defaults to [PROTOCOL_VERSION]. mailbox (int | list[tuple[bytes, int, bytes]] | None): If None, don't use mailbox. If int, use mailbox with this size in bytes. If list, use mailbox with this initial data. List should consist of OUI, type, data tuples.

failure_process(error_code: int) → None

Should be called when a failure state has occurred. returns an error code. Destroys all session bound keys and data.

handle_auth0(auth0_command: Command) → None

Parse a auth0 command and send the appropriate response.

Args:

auth0_command (Command): The command to respond to.

Raises:

SessionError: Raised when the session is missing or in an invalid state. VersionError: Raised when the protocol version is not supported. NotImplementedError:

handle_auth1(auth1_command: Command) → None

Parse a auth1 command and send the appropriate response.

Args:

auth1_command (Command): The command to respond to.

Raises:

SessionError: Raised when the session is missing or in an invalid state. KeyLookupFailed: When the reader public key cannot be found. AccessProtocolError: Raised if the response has invalid data.

handle_control_flow(control_flow_command: Command) → None

Parse an control flow command and send the appropriate response.

Args:

control_flow_command (Command): The command to respond to.

Raises:

SessionError: Raised when the session is missing or in an invalid state.

handle_exchange(exchange_command: Command) → None

Parse an exchange command and send the appropriate response.

Args:

exchange_command (Command): The command to respond to.

Raises:

SessionError: Raised when the session is missing or in an invalid state. AccessProtocolError: Raised if the response has invalid data.

handle_load_cert(load_cert_command: Command) → None

Parse a load cert command and send the appropriate response.

Args:

load_cert_command (Command): The command to respond to.

Raises:

SessionError: Raised when the session is missing or in an invalid state. KeyLookupFailed: When the reader public key cannot be found.

handle_select(select_command: Command) → bytes

Parse a select command and send the appropriate response.

Args:

select_command (Command): The command to respond to.

Raises:

SessionError: Raised if no session is found. InvalidAIDError: Raised when the AID is invalid.

main_loop() → None

Starts a loop, where every command received is replied with an appropriate response. Should keep running, even when receiving invalid commands.

Raises:

SessionError: When starting a new session failed. NotImplementedError: When a command which is not implemented is received.

response_auth0(*credential_epubk*: bytes, *cryptogram*: bytes | None = None) → None

Create and send an auth0 response.

Args:

credential_epubk (bytes): Credential Ephemeral public key. *cryptogram* (bytes | None, optional): authentication cryptogram. Defaults to None.

response_auth1(*key_slot*: bytes | None, *credential_public_key*: bytes | None, *expected_response*: Auth1Response, *signature*: bytes, *encryption*: EncryptionEngine, *status*: int = StatusBytes.SUCCESS, *private_mailbox_data*: bytes | None = None, *access_doc_retrieve*: bool = False, *revocation_doc_retrieve*: bool = False, *step_up_aid_required*: bool = False, *data_in_mailbox*: bool = False, *read_mailbox*: bool = False, *write_mailbox*: bool = False, *send_issuer_backend*: bool = False, *send_bound_app*: bool = False, *update_doc*: bool = False, *credential_signed_timestamp*: bytes | None = None, *revocation_signed_timestamp*: bytes | None = None) → None

Create and send an auth1 response.

Args:

key_slot (bytes | None): First 8 bytes of the keyIdentifier. *credential_public_key* (bytes | None): Credential long term public key. *expected_response* (Auth1Response): expected response (keyslot or credential public key) *signature* (bytes): User device authentication signature. *encryption* (EncryptionEngine): Encryption engine to encrypt the response. *status* (int, optional): response status. Defaults to StatusBytes.SUCCESS. *private_mailbox_data* (bytes | None, optional): Defaults to None. *access_doc_retrieve* (bool, optional): Part of signaling bitmap. Defaults to False. *revocation_doc_retrieve* (bool, optional): Part of signaling bitmap. Defaults to False. *step_up_aid_required* (bool, optional): Part of signaling bitmap. Defaults to False. *data_in_mailbox* (bool, optional): Part of signaling bitmap. Defaults to False. *read_mailbox* (bool, optional): Part of signaling bitmap. Defaults to False. *write_mailbox* (bool, optional): Part of signaling bitmap. Defaults to False. *send_issuer_backend* (bool, optional): Part of signaling bitmap. Defaults to False. *send_bound_app* (bool, optional): Part of signaling bitmap. Defaults to False. *update_doc* (bool, optional): Part of signaling bitmap. Defaults to False. *credential_signed_timestamp* (bytes | None, optional): Defaults to None. *revocation_signed_timestamp* (bytes | None, optional): Defaults to None.

response_control_flow() → None

Create and send a control flow response.

response_envelope() → None

response_exchange(*payload*: bytes, *encryption*: EncryptionEngine) → None

Create and send a exchange response.

Args:

payload (bytes): exchange response payload. *encryption* (EncryptionEngine): encryption engine to encrypt the payload.

response_get_response() → None

response_load_cert() → None

Create and send a load cert response.

response_select(*aid*: bytes, *type*: int, *protocol_versions*: list[int], *maximum_command_apdu*: int | None = None, *maximum_response_apdu*: int | None = None, *vendor_specific_tlv*: TLV | None = None) → None

Create and send a select response.

Args:

aid (bytes): AID type (int): Application type (table 10-4) *protocol_versions* (list[int]): Expedited

phase supported protocol versions. `maximum_command_apdu` (int | None, optional): Defaults to None. `maximum_response_apdu` (int | None, optional): Defaults to None. `vendor_specific_tlv` (TLV | None, optional): Defaults to None.

return_exchange_error_and_close_channel() → None

Return an exchange error and close the channel. Used when an exchange fails.

Raises:

`SessionError`: Raised if no session is found. `AccessProtocolError`: Raised if no encryption engine is found.

start_new_session(*ephemeral_key*: `KeyPair` | None = None) → None

Start a new user session. Must be done before using handle commands. This sessions stores all information received from commands. Start a new session to delete all received info and start over.

Args:

`ephemeral_key` (`KeyPair` | None, optional): ephemeral reader key used for the session. Randomly generated if None. Defaults to None.

transaction_initiation() → None

Initializes the hardware and sets up a connection to the reader.

wait_for_command(*expected_command*: `INS` | list[`INS`] | None = None, *encryption*: `EncryptionEngine` | None = None) → `Command`

Waits until a command is received, and parses the command.

Args:

`expected_command` (`INS` | list[`INS`] | None, optional): `INS` or list of `INS` with expected commands. raises `UnexpectedCommandError` if another command is received. Defaults to None. `encryption` (`EncryptionEngine` | None, optional): Used for decrypting messages. Not required for every command. Defaults to None.

Raises:

`InvalidCLAError`: Raised when the received command has an invalid CLA. `InvalidParameterError`: Raised when the received command has an invalid Parameter (P1 or P2). `InvalidCommandError`: Raised when the received command is invalid. `VerificationError`: Raised when the verification of an AES decryption fails. `UnexpectedCommandError`: when the command is not in `expected_command`.

Returns:

`Command`: the received command.

```
class aliro_actuator.access_protocol.user_device.UserSession(supported_version: list[int],
                                                             vendor_extension: bytes | None =
                                                             None)
```

Bases: object

Contains info from a single session (with one Reader Device)

derive_key_persistent(*transport_protocol*: `TransportProtocol`) → None

derive_key_volatile(*transport_protocol*: `TransportProtocol`) → None

generate_ephemeral_key(*ephemeral_key*: `KeyPair` | None = None) → None

get_credential_epubkey() → `PublicKey`

get_reader_public_key() → `PublicKey`

get_transaction_type() → `Transaction`

```
property reader_group_identifier: bytes
property reader_group_sub_identifier: bytes
property reader_identifier: bytes
set_access_credential(access_credential: AccessCredential) → None
set_auth0_data(auth0_command: Command) → None
set_cert_and_verify(compressed_cert: bytes, public_key: PublicKey) → bool
set_shared_key() → None
state_valid(state: list[UserSessionState] | UserSessionState) → bool
update_state(state: UserSessionState) → None
```

```
class aliro_actuator.access_protocol.user_device.UserSessionState(value)
```

Bases: Enum

An enumeration.

```
AUTH0_FAST_DONE = 4
```

```
AUTH0_STD_DONE = 3
```

```
AUTH1_DONE = 5
```

```
EXCHANGE_DONE = 6
```

```
SELECT_DONE = 2
```

```
SESSION_START = 1
```

Module contents

Access Protocol

Implements the access protocol part of the aliro protocol

```
class aliro_actuator.access_protocol.Device(transport_protocol: TransportProtocol, transport_override:
                                             TransportProtocolBase | None = None)
```

Bases: object

```
property support_extended_length_apdu: bool
```

aliro_actuator.hw_driver package

Subpackages

aliro_actuator.hw_driver.pn7160_driver package

Submodules

aliro_actuator.hw_driver.pn7160_driver.api module

```

class aliro_actuator.hw_driver.pn7160_driver.api.TECHNOLOGY_MASK(value)
    Bases: IntEnum
    An enumeration.
    DEFAULT = -1
    MASK_A = 1

class aliro_actuator.hw_driver.pn7160_driver.api.ndef_info_t
    Bases: Structure
    current_ndef_length
        Structure/Union member
    is_def
        Structure/Union member
    is_writable
        Structure/Union member
    max_ndef_length
        Structure/Union member

class aliro_actuator.hw_driver.pn7160_driver.api.nfcHostCardEmulationCallback_t
    Bases: Structure
    onDataReceived
        Structure/Union member
    onHostCardEmulationActivated
        Structure/Union member
    onHostCardEmulationDeactivated
        Structure/Union member

class aliro_actuator.hw_driver.pn7160_driver.api.nfcTagCallback_t
    Bases: Structure
    onTagArrival
        Structure/Union member
    onTagDeparture
        Structure/Union member

class aliro_actuator.hw_driver.pn7160_driver.api.nfc_tag_info_t
    Bases: Structure
    handle
        Structure/Union member
    protocol
        Structure/Union member
    technology
        Structure/Union member

```

uid

Structure/Union member

uid_length

Structure/Union member

aliro_actuator.hw_driver.pn7160_driver.errors module

exception aliro_actuator.hw_driver.pn7160_driver.errors.**DriverNotInitializedError**(*error_message: str | None = None*)

Bases: [PN7160DriverError](#)

exception aliro_actuator.hw_driver.pn7160_driver.errors.**NCIError**(*error_code: int, error_message: str | None = None*)

Bases: [PN7160DriverError](#)

exception aliro_actuator.hw_driver.pn7160_driver.errors.**NCINotFoundError**

Bases: [PN7160DriverError](#)

exception aliro_actuator.hw_driver.pn7160_driver.errors.**NoDataReceivedError**

Bases: [PN7160DriverError](#)

exception aliro_actuator.hw_driver.pn7160_driver.errors.**NoReaderError**(*error_message: str | None = None*)

Bases: [PN7160DriverError](#)

exception aliro_actuator.hw_driver.pn7160_driver.errors.**NoTagError**(*error_message: str | None = None*)

Bases: [PN7160DriverError](#)

exception aliro_actuator.hw_driver.pn7160_driver.errors.**PN7160DriverError**

Bases: Exception

Module contents

class aliro_actuator.hw_driver.pn7160_driver.**Driver**(*nci_location: str | None = None*)

Bases: object

initialize(*mode: Mode*) → None

receive_message() → bytes

send_message(*message: bytes*) → None

wait_for_reader() → None

wait_for_tag() → None

Module contents

aliro_actuator.transport_protocol package

Submodules

aliro_actuator.transport_protocol.ble_uwb module

class aliro_actuator.transport_protocol.ble_uwb.BLEUWB

Bases: *TransportProtocolBase*

get_message() → bytes

initialization(mode: Mode) → None

send_message(command: bytes) → None

wait_for_connection() → None

aliro_actuator.transport_protocol.errors module

exception aliro_actuator.transport_protocol.errors.InvalidModeError

Bases: *TransportProtocolError*

exception aliro_actuator.transport_protocol.errors.NoDataReceivedError

Bases: *TransportProtocolError*

exception aliro_actuator.transport_protocol.errors.TransportProtocolError

Bases: Exception

aliro_actuator.transport_protocol.nfc module

class aliro_actuator.transport_protocol.nfc.NFC(port: str | None = None)

Bases: *TransportProtocolBase*

get_message() → bytes

initialization(mode: Mode) → None

send_message(command: bytes) → None

wait_for_connection() → None

aliro_actuator.transport_protocol.socket module

class aliro_actuator.transport_protocol.socket.**Socket**

Bases: *TransportProtocolBase*

Uses sockets to communicate. Mainly used for testing purposes.

disconnect() → None

get_message() → bytes

initialization(mode: Mode) → None

send_message(command: bytes) → None

wait_for_connection() → None

Module contents

class aliro_actuator.transport_protocol.**Mode**(value)

Bases: Enum

An enumeration.

CARD_EMULATION = 2

READER = 1

class aliro_actuator.transport_protocol.**TransportProtocolBase**

Bases: ABC

abstract get_message() → bytes

abstract initialization(mode: Mode) → None

abstract send_message(command: bytes) → None

abstract wait_for_connection() → None

aliro_actuator.trust_framework package

Submodules

aliro_actuator.trust_framework.access_credential module

class aliro_actuator.trust_framework.access_credential.**AccessCredential**(user_device_key_pair: *KeyPair*,
reader_id_key_list: *list[tuple[bytes, PublicKey]]*)

Bases: object

summary

Args:

`user_device_key_pair` (KeyPair): keypair used by the User Device when this AccessCredential has a matching reader group identifier
`reader_id_key_list` (list[tuple[bytes, PublicKey]]): a list with tuples containing reader_group_identifier and reader_public_key pairs

get_credential_public_key() → *PublicKey*

get_key_slot() → bytes

get_reader_public_key(identifier: bytes) → *PublicKey*

has_identifier(group_identifier: bytes) → bool

Checks if this AccessCredential has the given reader group identifier, and the reader public key can be used for this reader device.

Args:

`group_identifier` (bytes): reader group identifier to check.

Returns:

bool: True if this access_credential has the given reader group identifier.

sign(data: bytes) → bytes

aliro_actuator.trust_framework.certificate module

```
class aliro_actuator.trust_framework.certificate.Certificate(version: bytes = b'', serial_number:
    bytes = b'', signature: bytes = b'',
    issuer: bytes = b'',
    validity_not_before: bytes = b'',
    validity_not_after: bytes = b'',
    subject: bytes = b'',
    key_info_algorithm: bytes = b'',
    key_info_parameters: bytes = b'',
    key_info_subject_public_key: bytes
    = b'', authority_key_identifier: bytes
    = b'', key_usage_extension: bytes =
    b'', signature_algorithm: bytes = b'',
    signature_value: bytes = b'')
```

Bases: object

classmethod decode(certificate: bytes) → *Certificate*

classmethod decode_compressed(compressed_certificate: bytes) → *Certificate*

default_issuer = b'issuer'

default_serial_number = b'\x01'

default_subject = b'subject'

default_validity_not_after = b'490101000000Z'

default_validity_not_before = b'200101000000Z'

encode_compressed() → bytes

get_public_key() → *PublicKey*

verify(*key*: [PublicKey](#), *data*: *bytes*) → bool

aliro_actuator.trust_framework.errors module

exception `aliro_actuator.trust_framework.errors.CertificateDecodingError`(*certificate*: *bytes*,
message: *str* | *None* = *None*)

Bases: [TrustFrameworkError](#)

Passed when a certificate cannot be decoded.

exception `aliro_actuator.trust_framework.errors.InvalidIdentifierError`(*identifier*: *bytes*,
message: *str* | *None* = *None*)

Bases: [TrustFrameworkError](#)

Passed when a identifier is invalid.

exception `aliro_actuator.trust_framework.errors.InvalidKeyError`(*key*: *bytes*, *message*: *str* | *None* = *None*)

Bases: [TrustFrameworkError](#)

Passed when a key is invalid.

exception `aliro_actuator.trust_framework.errors.InvalidKeyFormatError`

Bases: [TrustFrameworkError](#)

Passed when a key has the wrong format.

exception `aliro_actuator.trust_framework.errors.KeyLookupFailed`

Bases: [TrustFrameworkError](#)

exception `aliro_actuator.trust_framework.errors.MissingPublicKeyError`

Bases: [TrustFrameworkError](#)

Passed when initializing private key with 32 bytes data, and no public key.

exception `aliro_actuator.trust_framework.errors.TrustFrameworkError`

Bases: *Exception*

Parent class of all errors related to the trust framework.

aliro_actuator.trust_framework.key module

class `aliro_actuator.trust_framework.key.Key`

Bases: *object*

Base class for all keys.

class `aliro_actuator.trust_framework.key.KeyPair`(*private_key*: *str* | *bytes* | [PrivateKey](#) | *None* = *None*,
public_key: *bytes* | *str* | [PublicKey](#) | *None* = *None*)

Bases: *object*

Combination of a private and public key.

get_private_key() → [PrivateKey](#)

Returns the private key.

get_public_key() → *PublicKey*

Returns the public key.

get_public_key_as_bytes() → bytes

Returns the public key as raw bytes. Prepend with 0x04, total length of 65.

sign(data: bytes) → bytes

Sign the data. The signature is returned.

(figure 8-5 of Aliro spec)

verify(data: bytes, signature: bytes) → bool

Verifies the data. Returns True when the verification succeeds.

(figure 8-6 of Aliro spec)

class aliro_actuator.trust_framework.key.**PrivateKey**(key: bytes | str | None = None, public_key: bytes | None = None)

Bases: *Key*

Private Key

as_bytes(short: bool = True) → bytes

Returns the key as raw bytes.

as_pem() → str

Returns the key as PEM.

compute_shared_key(public_key: *PublicKey*, shared_info: bytes) → bytes

Compute a shared key using Diffie-Hellman

(figure 8-7 of the Aliro spec)

generate_public_key() → *PublicKey*

Generate a public key from this private key.

sign(data: bytes) → bytes

Sign the data. The signature is returned.

(figure 8-5 of Aliro spec)

class aliro_actuator.trust_framework.key.**PublicKey**(key: bytes | *EllipticCurvePublicKey* | str)

Bases: *Key*

Public Key

as_bytes() → bytes

Returns the key as raw bytes. Prepend with 0x04, total length of 65.

as_pem() → str

Returns the key as PEM.

get_x() → int

get_y() → int

verify(data: bytes, signature: bytes) → bool

Verifies the data. Returns True when the verification succeeds.

(figure 8-6 of Aliro spec)

`aliro_actuator.trust_framework.key.derive_key(input_key: bytes, info: bytes, length: int, salt: bytes) → bytes`

Derive key

(figure 8-8 of the Aliro spec)

`aliro_actuator.trust_framework.reader_identifier` module

`class aliro_actuator.trust_framework.reader_identifier.ReaderIdentifier(identifier: bytes)`

Bases: object

`as_bytes() → bytes`

`get_group() → bytes`

`get_group_sub() → bytes`

Module contents

3.1.1.2 Submodules

`aliro_actuator.log` module

`class aliro_actuator.log.InterceptHandler(log_source: str)`

Bases: Handler

This class is used to intercept the standard python logging. To enable it, you must call.

This is based on boilerplate code from the loguru project

`logging.basicConfig(handlers=[InterceptHandler()], level=0)`

`emit(record: LogRecord) → None`

This will be called by the standard logging module on each log call

Args:

record (logging.LogRecord): The log record intercepted from the standard logging module

`loglevel_mapping = {0: 'NOTSET', 10: 'DEBUG', 20: 'INFO', 30: 'WARNING', 40: 'ERROR', 50: 'CRITICAL'}`

`exception aliro_actuator.log.LoggingConfigError`

Bases: Exception

Raised when config parameters are missing or don't align

`aliro_actuator.log.configure_logging() → None`

3.1.1.3 Module contents

```
class aliro_actuator.Global
```

```
    Bases: object
```

```
    logger = <loguru.logger handlers=[(id=1, level=10, sink=<stdout>), (id=2, level=10, sink='/app/src/./logs/{time:YYYY-MM-DD--HH-mm-ss}.log')]>
```

```
    classmethod set_logger(logger) → None
```


INDICES AND TABLES

- genindex
- modindex
- search

PYTHON MODULE INDEX

a

- `aliro_actuator`, 41
- `aliro_actuator.access_document`, 8
- `aliro_actuator.access_document.access_credential`, 7
- `aliro_actuator.access_document.errors`, 7
- `aliro_actuator.access_document.revocation_document`, 8
- `aliro_actuator.access_protocol`, 32
- `aliro_actuator.access_protocol.apdu`, 8
- `aliro_actuator.access_protocol.authentication`, 13
- `aliro_actuator.access_protocol.defines`, 14
- `aliro_actuator.access_protocol.encryption`, 17
- `aliro_actuator.access_protocol.errors`, 19
- `aliro_actuator.access_protocol.mailbox`, 20
- `aliro_actuator.access_protocol.reader`, 22
- `aliro_actuator.access_protocol.tlv`, 26
- `aliro_actuator.access_protocol.user_device`, 28
- `aliro_actuator.hw_driver`, 35
- `aliro_actuator.hw_driver.pn7160_driver`, 34
- `aliro_actuator.hw_driver.pn7160_driver.api`, 33
- `aliro_actuator.hw_driver.pn7160_driver.errors`, 34
- `aliro_actuator.log`, 40
- `aliro_actuator.transport_protocol`, 36
- `aliro_actuator.transport_protocol.ble_uwb`, 35
- `aliro_actuator.transport_protocol.errors`, 35
- `aliro_actuator.transport_protocol.nfc`, 35
- `aliro_actuator.transport_protocol.socket`, 36
- `aliro_actuator.trust_framework`, 40
- `aliro_actuator.trust_framework.access_credential`, 36
- `aliro_actuator.trust_framework.certificate`, 37
- `aliro_actuator.trust_framework.errors`, 38
- `aliro_actuator.trust_framework.key`, 38
- `aliro_actuator.trust_framework.reader_identifier`, 40

INDEX

A

AccessCredential (class in module, 22
 aliro_actuator.trust_framework.access_credential),
 36
 AccessDocument (class in module, 28
 aliro_actuator.access_document.access_credential),
 7
 AccessProtocolError, 19
 add_set() (*aliro_actuator.access_protocol.user_device.MailboxSession*
 method), 28
 add_value() (*aliro_actuator.access_protocol.tlv.TLV*
 method), 26
 add_write() (*aliro_actuator.access_protocol.user_device.MailboxSession*
 method), 28
 AID_LEN (*aliro_actuator.access_protocol.defines.Select*
 attribute), 16
 AID_TAG (*aliro_actuator.access_protocol.defines.Select*
 attribute), 16
 aliro_actuator
 module, 41
 aliro_actuator.access_document
 module, 8
 aliro_actuator.access_document.access_credential
 module, 7
 aliro_actuator.access_document.errors
 module, 7
 aliro_actuator.access_document.revocation_document
 module, 8
 aliro_actuator.access_protocol
 module, 32
 aliro_actuator.access_protocol.apdu
 module, 8
 aliro_actuator.access_protocol.authentication
 module, 13
 aliro_actuator.access_protocol.defines
 module, 14
 aliro_actuator.access_protocol.encryption
 module, 17
 aliro_actuator.access_protocol.errors
 module, 19
 aliro_actuator.access_protocol.mailbox
 module, 20
 aliro_actuator.access_protocol.reader
 module, 22
 aliro_actuator.access_protocol.tlv
 module, 26
 aliro_actuator.access_protocol.user_device
 module, 28
 aliro_actuator.hw_driver
 module, 35
 aliro_actuator.hw_driver.pn7160_driver
 module, 34
 aliro_actuator.hw_driver.pn7160_driver.api
 module, 33
 aliro_actuator.hw_driver.pn7160_driver.errors
 module, 34
 aliro_actuator.log
 module, 40
 aliro_actuator.transport_protocol
 module, 36
 aliro_actuator.transport_protocol.ble_uwb
 module, 35
 aliro_actuator.transport_protocol.errors
 module, 35
 aliro_actuator.transport_protocol.nfc
 module, 35
 aliro_actuator.transport_protocol.socket
 module, 36
 aliro_actuator.trust_framework
 module, 40
 aliro_actuator.trust_framework.access_credential
 module, 36
 aliro_actuator.trust_framework.certificate
 module, 37
 aliro_actuator.trust_framework.errors
 module, 38
 aliro_actuator.trust_framework.key
 module, 38
 aliro_actuator.trust_framework.reader_identifier
 module, 40
 APDU (class in *aliro_actuator.access_protocol.apdu*), 8
 as_bytes() (*aliro_actuator.trust_framework.key.PrivateKey*
 method), 39
 as_bytes() (*aliro_actuator.trust_framework.key.PublicKey*

method), 39

as_bytes() (aliro_actuator.trust_framework.reader_identifier.ReaderIdentifier method), 40

as_pem() (aliro_actuator.trust_framework.key.PrivateKey method), 39

as_pem() (aliro_actuator.trust_framework.key.PublicKey method), 39

AUTH0 (aliro_actuator.access_protocol.apdu.INS attribute), 10

Auth0 (class in aliro_actuator.access_protocol.defines), 14

AUTH0_FAST_DONE (aliro_actuator.access_protocol.user_device.UserSession attribute), 32

AUTH0_STD_DONE (aliro_actuator.access_protocol.user_device.UserSession attribute), 32

AUTH1 (aliro_actuator.access_protocol.apdu.INS attribute), 10

Auth1 (class in aliro_actuator.access_protocol.defines), 14

AUTH1_DONE (aliro_actuator.access_protocol.user_device.UserSession attribute), 32

Auth1Response (class in aliro_actuator.access_protocol.apdu), 9

B

BLE_UWB (aliro_actuator.access_protocol.defines.TransportProtocol attribute), 16

BLEUWB (class in aliro_actuator.transport_protocol.ble_uwb), 35

C

can_retrieve_access_credential() (aliro_actuator.access_protocol.reader.ReaderSession method), 25

can_retrieve_revocation_document() (aliro_actuator.access_protocol.reader.ReaderSession method), 25

CARD_EMULATION (aliro_actuator.transport_protocol.Mode attribute), 36

Certificate (class in aliro_actuator.trust_framework.certificate), 37

CERTIFICATE_TAG (aliro_actuator.access_protocol.defines.Auth1 attribute), 15

CertificateDecodingError, 38

check_boundaries() (aliro_actuator.access_protocol.mailbox.Mailbox method), 20

check_counters_valid() (aliro_actuator.access_protocol.encryption.EncryptionEngine method), 17

check_user_device_authentication() (aliro_actuator.access_protocol.reader.ReaderSession method), 25

Command (class in aliro_actuator.access_protocol.apdu), 9

Identifier (class in aliro_actuator.access_protocol.apdu), 9

command_auth0() (aliro_actuator.access_protocol.reader.Reader method), 22

command_auth1() (aliro_actuator.access_protocol.reader.Reader method), 22

command_control_flow() (aliro_actuator.access_protocol.reader.Reader method), 22

command_envelope() (aliro_actuator.access_protocol.reader.Reader method), 23

command_exchanger() (aliro_actuator.access_protocol.reader.Reader method), 23

command_get_response() (aliro_actuator.access_protocol.reader.Reader method), 23

COMMAND_LEN (aliro_actuator.access_protocol.defines.Auth0 attribute), 14

COMMAND_LEN (aliro_actuator.access_protocol.defines.Auth1 attribute), 15

command_load_cert() (aliro_actuator.access_protocol.reader.Reader method), 23

COMMAND_NOT_ALLOWED (aliro_actuator.access_protocol.apdu.StatusBytes attribute), 12

COMMAND_NOT_COMPLIANT (aliro_actuator.access_protocol.apdu.StatusBytes attribute), 12

command_select() (aliro_actuator.access_protocol.reader.Reader method), 23

COMMAND_TAG (aliro_actuator.access_protocol.defines.Auth0 attribute), 14

COMMAND_TAG (aliro_actuator.access_protocol.defines.Auth1 attribute), 15

compute_shared_key() (aliro_actuator.trust_framework.key.PrivateKey method), 39

CONDITIONS_OF_USE_NOT_SATISFIED (aliro_actuator.access_protocol.apdu.StatusBytes attribute), 12

configure_logging() (in module aliro_actuator.log), 40

CONTROL_FLOW (aliro_actuator.access_protocol.apdu.INS attribute), 11

ControlFlow (class in aliro_actuator.access_protocol.defines), 15

create_auth0_command() (aliro_actuator.access_protocol.apdu.APDU method), 8

create_auth0_response() (aliro_actuator.access_protocol.apdu.APDU method), 8

create_auth1_command()

`(aliro_actuator.access_protocol.apdu.APDU method), 8`
`create_auth1_response()`
`(aliro_actuator.access_protocol.apdu.APDU method), 8`
`create_command()` `(aliro_actuator.access_protocol.apdu.APDU method), 8`
`create_control_flow_command()`
`(aliro_actuator.access_protocol.apdu.APDU method), 9`
`create_control_flow_response()`
`(aliro_actuator.access_protocol.apdu.APDU method), 9`
`create_envelope_command()`
`(aliro_actuator.access_protocol.apdu.APDU method), 9`
`create_envelope_response()`
`(aliro_actuator.access_protocol.apdu.APDU method), 9`
`create_error_response()`
`(aliro_actuator.access_protocol.apdu.APDU method), 9`
`create_exchange_command()`
`(aliro_actuator.access_protocol.apdu.APDU method), 9`
`create_exchange_response()`
`(aliro_actuator.access_protocol.apdu.APDU method), 9`
`create_from_bytestring()`
`(aliro_actuator.access_protocol.apdu.Command class method), 9`
`create_from_bytestring()`
`(aliro_actuator.access_protocol.apdu.Response class method), 11`
`create_from_parameters()`
`(aliro_actuator.access_protocol.apdu.Command class method), 10`
`create_from_parameters()`
`(aliro_actuator.access_protocol.apdu.Response class method), 11`
`create_get_response_command()`
`(aliro_actuator.access_protocol.apdu.APDU method), 9`
`create_get_response_response()`
`(aliro_actuator.access_protocol.apdu.APDU method), 9`
`create_load_cert_command()`
`(aliro_actuator.access_protocol.apdu.APDU method), 9`
`create_load_cert_response()`
`(aliro_actuator.access_protocol.apdu.APDU method), 9`
`create_proprietary_information()` `(in module aliro_actuator.access_protocol.encryption), 18`
`create_reader_authentication()` `(in module aliro_actuator.access_protocol.authentication), 13`
`create_salt()` `(in module aliro_actuator.access_protocol.encryption), 18`
`create_select_command()`
`(aliro_actuator.access_protocol.apdu.APDU method), 9`
`create_select_response()`
`(aliro_actuator.access_protocol.apdu.APDU method), 9`
`create_user_device_authentication()` `(in module aliro_actuator.access_protocol.authentication), 13`
`CreateCommandError, 19`
`CreateResponseError, 19`
`CREDENTIAL_EPUBK_LEN`
`(aliro_actuator.access_protocol.defines.Auth0 attribute), 14`
`CREDENTIAL_EPUBK_TAG`
`(aliro_actuator.access_protocol.defines.Auth0 attribute), 14`
`CREDENTIAL_EPUBK_TAG`
`(aliro_actuator.access_protocol.defines.ReaderAuth attribute), 16`
`CREDENTIAL_EPUBK_TAG`
`(aliro_actuator.access_protocol.defines.UserDeviceAuth attribute), 16`
`CREDENTIAL_PUBK_LEN`
`(aliro_actuator.access_protocol.defines.Auth1 attribute), 15`
`CREDENTIAL_PUBK_TAG`
`(aliro_actuator.access_protocol.defines.Auth1 attribute), 15`
`CREDENTIAL_PUBLIC_KEY`
`(aliro_actuator.access_protocol.apdu.Auth1Response attribute), 9`
`CREDENTIAL_TIMESTAMP_LEN`
`(aliro_actuator.access_protocol.defines.Auth1 attribute), 15`
`CREDENTIAL_TIMESTAMP_TAG`
`(aliro_actuator.access_protocol.defines.Auth1 attribute), 15`
`CRYPTOGRAM_LEN` `(aliro_actuator.access_protocol.defines.Auth0 attribute), 14`
`CRYPTOGRAM_TAG` `(aliro_actuator.access_protocol.defines.Auth0 attribute), 14`
`current_ndef_length`
`(aliro_actuator.hw_driver.pn7160_driver.api.ndef_info_t attribute), 33`
D
`data_is_set()` `(aliro_actuator.access_protocol.mailbox.Mailbox method), 20`

DATA_TAG (aliro_actuator.access_protocol.mailbox.Mailbox attribute), 20

data_element (class in aliro_actuator.access_document.access_credential), 7

decode() (aliro_actuator.trust_framework.certificate.Certificate class method), 37

decode_compressed() (aliro_actuator.trust_framework.certificate.Certificate class method), 37

decrypt() (aliro_actuator.access_protocol.encryption.EncryptionEngine method), 17

decrypt_payload() (aliro_actuator.access_protocol.reader.ReaderSession method), 25

DEFAULT (aliro_actuator.hw_driver.pn7160_driver.api.TECHNOLOGY_MASK attribute), 33

default_issuer (aliro_actuator.trust_framework.certificate.Certificate attribute), 37

default_serial_number (aliro_actuator.trust_framework.certificate.Certificate attribute), 37

default_subject (aliro_actuator.trust_framework.certificate.Certificate attribute), 37

default_validity_not_after (aliro_actuator.trust_framework.certificate.Certificate attribute), 37

default_validity_not_before (aliro_actuator.trust_framework.certificate.Certificate attribute), 37

derive_key() (in module aliro_actuator.trust_framework.key), 39

derive_key_persistent() (aliro_actuator.access_protocol.user_device.UserSession method), 31

derive_key_volatile() (aliro_actuator.access_protocol.reader.ReaderSession method), 25

derive_key_volatile() (aliro_actuator.access_protocol.user_device.UserSession method), 31

Device (class in aliro_actuator.access_protocol), 32

DeviceType (class in aliro_actuator.access_protocol.encryption), 17

disconnect() (aliro_actuator.transport_protocol.socket.Socket method), 36

Document (class in aliro_actuator.access_document), 8

DocumentError, 7

DOMAIN_SPECIFIC_TAG (aliro_actuator.access_protocol.defines.ControlFlow attribute), 15

Driver (class in aliro_actuator.hw_driver.pn7160_driver), 34

DriverNotInitializedError, 34

ElementSizeError, 7

emit() (aliro_actuator.log.InterceptHandler method), 40

encode_compressed() (aliro_actuator.trust_framework.certificate.Certificate method), 17

encrypt() (aliro_actuator.access_protocol.encryption.EncryptionEngine method), 17

encrypt_payload() (aliro_actuator.access_protocol.reader.ReaderSession method), 25

EncryptionEngine (class in aliro_actuator.access_protocol.encryption), 17

ENVELOPE (aliro_actuator.access_protocol.apdu.INS attribute), 11

ETPV_LEN (aliro_actuator.access_protocol.defines.Auth0 attribute), 14

ETPV_TAG (aliro_actuator.access_protocol.defines.Auth0 attribute), 14

ETSPV_TAG (aliro_actuator.access_protocol.defines.Select attribute), 16

EXCHANGE (aliro_actuator.access_protocol.apdu.INS attribute), 11

Exchange (class in aliro_actuator.access_protocol.defines), 15

EXCHANGE_DONE (aliro_actuator.access_protocol.user_device.UserSession attribute), 32

execute_commands() (aliro_actuator.access_protocol.user_device.Mailbox method), 28

expedited_transaction_fast() (aliro_actuator.access_protocol.reader.Reader method), 23

expedited_transaction_standard() (aliro_actuator.access_protocol.reader.Reader method), 23

EXTENDED_INFO_LEN (aliro_actuator.access_protocol.defines.Select attribute), 16

EXTENDED_INFO_TAG (aliro_actuator.access_protocol.defines.Select attribute), 16

F

failure_process() (aliro_actuator.access_protocol.reader.Reader method), 23

failure_process() (aliro_actuator.access_protocol.user_device.UserDevice method), 28

FAST (aliro_actuator.access_protocol.apdu.Transaction attribute), 13

FCI_TAG (aliro_actuator.access_protocol.defines.Select attribute), 16

FILE_OR_APP_NOT_FOUND (aliro_actuator.access_protocol.apdu.StatusBytes attribute), 12

FINISHED_WITH_FAILURE (aliro_actuator.access_protocol.apdu.S1 attribute), 12

FINISHED_WITH_SUCCESS
 (aliro_actuator.access_protocol.apdu.SI
 attribute), 12
 FORCE_USER_AUTHENTICATION
 (aliro_actuator.access_protocol.apdu.TransactionType
 attribute), 13
 from_bytes() (aliro_actuator.access_protocol.tlv.TLV
 static method), 26
 FUNCTION_NOT_SUPPORTED
 (aliro_actuator.access_protocol.apdu.StatusBytes
 attribute), 12
 FUNCTIONS_IN_CLA_NOT_SUPPORTED
 (aliro_actuator.access_protocol.apdu.StatusBytes
 attribute), 12
G
 generate_ephemeral_key()
 (aliro_actuator.access_protocol.reader.ReaderSession
 method), 25
 generate_ephemeral_key()
 (aliro_actuator.access_protocol.user_device.UserSession
 method), 31
 generate_public_key()
 (aliro_actuator.trust_framework.key.PrivateKey
 method), 39
 GENERIC_ERROR (aliro_actuator.access_protocol.apdu.StatusBytes
 attribute), 12
 get_all_bytes_of_tag()
 (aliro_actuator.access_protocol.tlv.TLV
 method), 26
 get_all_of_tag() (aliro_actuator.access_protocol.tlv.TLV
 method), 26
 get_bytes() (aliro_actuator.access_protocol.tlv.TLV
 method), 26
 get_credential_ephemeral_key()
 (aliro_actuator.access_protocol.reader.ReaderSession
 method), 25
 get_credential_epubkey()
 (aliro_actuator.access_protocol.user_device.UserSession
 method), 31
 get_credential_public_key()
 (aliro_actuator.trust_framework.access_credential.AccessCredential
 method), 37
 get_data() (aliro_actuator.access_protocol.mailbox.Mailbox
 method), 20
 get_group() (aliro_actuator.trust_framework.reader_identifier.ReaderIdentifier
 method), 40
 get_group_sub() (aliro_actuator.trust_framework.reader_identifier.ReaderIdentifier
 method), 40
 get_key_slot() (aliro_actuator.trust_framework.access_credential.AccessCredential
 method), 37
 get_message() (aliro_actuator.transport_protocol.ble_uwb.BLEUWB
 method), 35
 get_message() (aliro_actuator.transport_protocol.nfc.NFC
 method), 35
 get_message() (aliro_actuator.transport_protocol.socket.Socket
 method), 36
 get_message() (aliro_actuator.transport_protocol.TransportProtocolBase
 method), 36
 get_private_key() (aliro_actuator.trust_framework.key.KeyPair
 method), 38
 get_public_key() (aliro_actuator.trust_framework.certificate.Certificate
 method), 37
 get_public_key() (aliro_actuator.trust_framework.key.KeyPair
 method), 38
 get_public_key_as_bytes()
 (aliro_actuator.trust_framework.key.KeyPair
 method), 39
 get_raw() (aliro_actuator.access_protocol.mailbox.Mailbox
 method), 21
 get_reader_epubkey()
 (aliro_actuator.access_protocol.reader.ReaderSession
 method), 25
 get_reader_public_key()
 (aliro_actuator.access_protocol.user_device.UserSession
 method), 31
 get_reader_public_key()
 (aliro_actuator.trust_framework.access_credential.AccessCredential
 method), 37
 GET_RESPONSE (aliro_actuator.access_protocol.apdu.INS
 attribute), 11
 get_tlv() (aliro_actuator.access_protocol.mailbox.Mailbox
 method), 21
 get_tlv() (aliro_actuator.access_protocol.tlv.TLV
 method), 27
 get_transaction_type()
 (aliro_actuator.access_protocol.user_device.UserSession
 method), 31
 get_value() (aliro_actuator.access_protocol.tlv.TLV
 method), 27
 get_x() (aliro_actuator.trust_framework.key.PublicKey
 method), 39
 get_y() (aliro_actuator.trust_framework.key.PublicKey
 method), 39
 Global class in aliro_actuator, 41
H
 handle (aliro_actuator.hw_driver.pn7160_driver.api.nfc_tag_info_t
 method), 33
 handle_auth0() (aliro_actuator.access_protocol.reader.Reader
 method), 28
 handle_auth0() (aliro_actuator.access_protocol.user_device.UserDevice
 method), 28
 handle_auth1() (aliro_actuator.access_protocol.reader.Reader
 method), 23
 handle_auth1() (aliro_actuator.access_protocol.user_device.UserDevice
 method), 29

<code>handle_control_flow()</code> (<code>aliro_actuator.access_protocol.reader.Reader</code> method), 24	<code>InvalidINSError</code> , 19 <code>InvalidKeyError</code> , 38 <code>InvalidKeyFormatError</code> , 38
<code>handle_control_flow()</code> (<code>aliro_actuator.access_protocol.user_device.UserDevice</code> method), 29	<code>InvalidLcError</code> , 19 <code>InvalidLeError</code> , 19 <code>InvalidModeError</code> , 35
<code>handle_exchange()</code> (<code>aliro_actuator.access_protocol.reader.Reader</code> method), 24	<code>InvalidParameterError</code> , 19 <code>InvalidResponseDataError</code> , 19
<code>handle_exchange()</code> (<code>aliro_actuator.access_protocol.user_device.UserDevice</code> method), 29	<code>InvalidResponseError</code> , 19 <code>InvalidStatusError</code> , 20
<code>handle_load_cert()</code> (<code>aliro_actuator.access_protocol.reader.Reader</code> method), 24	<code>InvalidAliro_actuator.hw_driver.pn7160_driver.api.ndef_info_t</code> attribute), 33
<code>handle_load_cert()</code> (<code>aliro_actuator.access_protocol.user_device.UserDevice</code> method), 29	<code>InvalidMailboxSession</code> (<code>aliro_actuator.access_protocol.user_device.MailboxSession</code> method), 28
<code>handle_select()</code> (<code>aliro_actuator.access_protocol.reader.Reader</code> method), 24	<code>InvalidAliro_actuator.hw_driver.pn7160_driver.api.ndef_info_t</code> attribute), 33
<code>handle_select()</code> (<code>aliro_actuator.access_protocol.user_device.UserDevice</code> method), 29	<code>InvalidAuthData</code> (class in <code>aliro_actuator.access_document.access_credential</code>), 7
<code>has_identifier()</code> (<code>aliro_actuator.trust_framework.access_credential.AccessCredential</code> method), 37	K <code>Key</code> (class in <code>aliro_actuator.trust_framework.key</code>), 38 <code>KEY_SLOT</code> (<code>aliro_actuator.access_protocol.apdu.Auth1Response</code> attribute), 9 <code>KEY_SLOT_LEN</code> (<code>aliro_actuator.access_protocol.defines.Auth1</code> attribute), 15 <code>KEY_SLOT_TAG</code> (<code>aliro_actuator.access_protocol.defines.Auth1</code> attribute), 15
<code>INCORRECT_P1_P2</code> (<code>aliro_actuator.access_protocol.apdu.StatusBytes</code> attribute), 13	<code>KeyLookupFailed</code> , 38 <code>KeyPair</code> (class in <code>aliro_actuator.trust_framework.key</code>), 38
<code>INCORRECT_PARAMETERS_IN_DATA</code> (<code>aliro_actuator.access_protocol.apdu.StatusBytes</code> attribute), 13	L <code>LOAD_CERT_FAILED</code> (<code>aliro_actuator.access_protocol.apdu.INS</code> at- tribute), 11
<code>INCORRECT_SECURE_MESSAGING_DOS</code> (<code>aliro_actuator.access_protocol.apdu.StatusBytes</code> attribute), 13	<code>Logger</code> (<code>aliro_actuator.Global</code> attribute), 41 <code>LoggingConfigError</code> , 40
<code>INDEX_TAG</code> (<code>aliro_actuator.access_protocol.mailbox.Mailbox</code> attribute), 20	<code>LOGICAL_CHANNEL_NOT_SUPPORTED</code> (<code>aliro_actuator.access_protocol.apdu.StatusBytes</code> attribute), 13
<code>initialization()</code> (<code>aliro_actuator.transport_protocol.ble.Ble</code> method), 35	<code>loglevel_mapping</code> (<code>aliro_actuator.log.InterceptHandler</code> attribute), 40
<code>initialization()</code> (<code>aliro_actuator.transport_protocol.nfc.Nfc</code> method), 35	<code>lookup_credential_public_key()</code> (<code>aliro_actuator.access_protocol.reader.ReaderSession</code> method), 25
<code>initialization()</code> (<code>aliro_actuator.transport_protocol.socket.Socket</code> method), 36	M <code>Mailbox</code> (class in <code>aliro_actuator.access_protocol.mailbox</code>), 20
<code>initialization()</code> (<code>aliro_actuator.transport_protocol.TransportProtocolBase</code> method), 36	<code>MAILBOX_DATA_TAG</code> (<code>aliro_actuator.access_protocol.defines.Auth1</code> attribute), 15
<code>initialize()</code> (<code>aliro_actuator.hw_driver.pn7160_driver.Driver</code> method), 34	<code>MailboxFormatError</code> , 21 <code>MailboxPermissionError</code> , 21
<code>INS</code> (class in <code>aliro_actuator.access_protocol.apdu</code>), 10	
<code>InterceptHandler</code> (class in <code>aliro_actuator.log</code>), 40	
<code>INVALID_CLASS</code> (<code>aliro_actuator.access_protocol.apdu.StatusBytes</code> attribute), 13	
<code>INVALID_INSTRUCTION</code> (<code>aliro_actuator.access_protocol.apdu.StatusBytes</code> attribute), 13	
<code>InvalidAIDError</code> , 19	
<code>InvalidCLAError</code> , 19	
<code>InvalidCommandDataError</code> , 19	
<code>InvalidCommandError</code> , 19	
<code>InvalidIdentifierError</code> , 38	

MailboxSession (class in aliro_actuator.access_protocol.user_device), 28

MailboxSession.Command (class in aliro_actuator.access_protocol.user_device), 28

main_loop() (aliro_actuator.access_protocol.user_device.UserDevice method), 29

MASK_A (aliro_actuator.hw_driver.pn7160_driver.api.TECHNOLOGY_MASK attribute), 33

MAX_COMMAND_TAG (aliro_actuator.access_protocol.defines.Select attribute), 16

max_ndef_length (aliro_actuator.hw_driver.pn7160_driver.api.ndef_info_t attribute), 33

MAX_RESPONSE_TAG (aliro_actuator.access_protocol.defines.Select attribute), 16

MEMORY_FAILURE (aliro_actuator.access_protocol.apdu.StatusBytes attribute), 13

Message (class in aliro_actuator.access_protocol.apdu), 11

MessageTooLongError, 20

MissingPublicKeyError, 38

Mode (class in aliro_actuator.transport_protocol), 36

module

- aliro_actuator, 41
- aliro_actuator.access_document, 8
- aliro_actuator.access_document.access_credentials, 7
- aliro_actuator.access_document.errors, 7
- aliro_actuator.access_document.revocation_document, 8
- aliro_actuator.access_protocol, 32
- aliro_actuator.access_protocol.apdu, 8
- aliro_actuator.access_protocol.authentication, 13
- aliro_actuator.access_protocol.defines, 14
- aliro_actuator.access_protocol.encryption, 17
- aliro_actuator.access_protocol.errors, 19
- aliro_actuator.access_protocol.mailbox, 20
- aliro_actuator.access_protocol.reader, 22
- aliro_actuator.access_protocol.tlv, 26
- aliro_actuator.access_protocol.user_device, 28
- aliro_actuator.hw_driver, 35
- aliro_actuator.hw_driver.pn7160_driver, 34
- aliro_actuator.hw_driver.pn7160_driver.api, 33
- aliro_actuator.hw_driver.pn7160_driver.errors, 34
- aliro_actuator.log, 40

aliro_actuator.transport_protocol, 36

aliro_actuator.transport_protocol.ble_uwb, 35

aliro_actuator.transport_protocol.errors, 35

aliro_actuator.transport_protocol.nfc, 35

aliro_actuator.transport_protocol.socket, 36

aliro_actuator.trust_framework, 40

aliro_actuator.trust_framework.access_credential, 36

aliro_actuator.trust_framework.certificate, 36

aliro_actuator.trust_framework.errors, 38

aliro_actuator.trust_framework.key, 38

aliro_actuator.trust_framework.reader_identifier, 40

N

NCIError, 34

NCINotFoundError, 34

ndef_info_t (class in aliro_actuator.hw_driver.pn7160_driver.api), 33

NFC (aliro_actuator.access_protocol.defines.TransportProtocol attribute), 16

nfc_tag_info_t (class in aliro_actuator.hw_driver.pn7160_driver.api), 33

nfcHostCardEmulationCallback_t (class in aliro_actuator.hw_driver.pn7160_driver.api), 33

nfcTagCallback_t (class in aliro_actuator.hw_driver.pn7160_driver.api), 33

NO_PRECISE_DIAGNOSIS (aliro_actuator.access_protocol.apdu.StatusBytes attribute), 13

NoDataReceivedError, 34, 35

NONE (aliro_actuator.access_protocol.apdu.S2 attribute), 12

NoReaderError, 34

NoTagError, 34

NOTIFY_TAG (aliro_actuator.access_protocol.defines.Exchange attribute), 15

O

onDataReceived (aliro_actuator.hw_driver.pn7160_driver.api.nfcHostCardEmulation attribute), 33

onHostCardEmulationActivated (aliro_actuator.hw_driver.pn7160_driver.api.nfcHostCardEmulation attribute), 33

onHostCardEmulationDeactivated	parse_command() (aliro_actuator.access_protocol.apdu.APDU
(aliro_actuator.hw_driver.pn7160_driver.api.nfcHostCardEmulationCallback_t	multitap(), 19
attribute), 33	parse_response() (aliro_actuator.access_protocol.apdu.APDU
onTagArrival (aliro_actuator.hw_driver.pn7160_driver.api.nfcTagCallback_t)	attribute), 33
attribute), 33	PN7160DriverError, 34
onTagDeparture (aliro_actuator.hw_driver.pn7160_driver.api.nfcTagCallback_t)	PrivateKey (class in aliro_actuator.trust_framework.key),
attribute), 33	39
P	PROPRIETARY_TAG (aliro_actuator.access_protocol.defines.Select
	attribute), 16
parse() (aliro_actuator.access_document.access_credentials.AccessDocument	protocol (aliro_actuator.hw_driver.pn7160_driver.api.nfc_tag_info_t
method), 7	attribute), 33
parse_as_auth0() (aliro_actuator.access_protocol.apdu.Command)	PROTOCOL_VERSION_NOT_SUPPORTED
method), 10	(aliro_actuator.access_protocol.apdu.S2
parse_as_auth0() (aliro_actuator.access_protocol.apdu.Response	attribute), 12
method), 11	PublicKey (class in aliro_actuator.trust_framework.key),
parse_as_auth1() (aliro_actuator.access_protocol.apdu.Command)	39
method), 10	R
parse_as_auth1() (aliro_actuator.access_protocol.apdu.Response	RE_VENDOR_SPECIFIC_MAX_LEN
method), 11	(aliro_actuator.access_protocol.defines.Auth0
parse_as_control_flow()	attribute), 14
(aliro_actuator.access_protocol.apdu.Command	RE_VENDOR_SPECIFIC_TAG
method), 10	(aliro_actuator.access_protocol.defines.Auth0
parse_as_control_flow()	attribute), 14
(aliro_actuator.access_protocol.apdu.Response	read() (aliro_actuator.access_protocol.mailbox.Mailbox
method), 11	method), 21
parse_as_envelope()	READ_TAG (aliro_actuator.access_protocol.defines.Exchange
(aliro_actuator.access_protocol.apdu.Command	attribute), 15
method), 10	READER (aliro_actuator.access_protocol.encryption.DeviceType
parse_as_envelope()	attribute), 17
(aliro_actuator.access_protocol.apdu.Response	READER (aliro_actuator.transport_protocol.Mode
method), 11	attribute), 36
parse_as_exchange()	Reader (class in aliro_actuator.access_protocol.reader),
(aliro_actuator.access_protocol.apdu.Command	22
method), 10	READER_EPUBK_LEN (aliro_actuator.access_protocol.defines.Auth0
parse_as_exchange()	attribute), 14
(aliro_actuator.access_protocol.apdu.Response	READER_EPUBK_TAG (aliro_actuator.access_protocol.defines.Auth0
method), 11	attribute), 14
parse_as_get_response()	READER_EPUBK_TAG (aliro_actuator.access_protocol.defines.ReaderAuth
(aliro_actuator.access_protocol.apdu.Command	attribute), 16
method), 10	READER_EPUBK_TAG (aliro_actuator.access_protocol.defines.UserDeviceAu
parse_as_get_response()	attribute), 16
(aliro_actuator.access_protocol.apdu.Response	reader_group_identifier
method), 12	(aliro_actuator.access_protocol.reader.Reader
parse_as_load_cert()	property), 24
(aliro_actuator.access_protocol.apdu.Command	reader_group_identifier
method), 10	(aliro_actuator.access_protocol.reader.ReaderSession
parse_as_load_cert()	property), 25
(aliro_actuator.access_protocol.apdu.Response	reader_group_identifier
method), 12	(aliro_actuator.access_protocol.user_device.UserSession
parse_as_select() (aliro_actuator.access_protocol.apdu.Command)	property), 31
method), 10	reader_group_sub_identifier
parse_as_select() (aliro_actuator.access_protocol.apdu.Response)	(aliro_actuator.access_protocol.reader.Reader
method), 12	property), 24

set_access_credential() (aliro_actuator.access_protocol.user_device.UserSession method), 25
 set_auth0_data() (aliro_actuator.access_protocol.user_device.UserSession method), 32
 set_auth1_info() (aliro_actuator.access_protocol.reader.ReaderSession method), 25
 set_cert_and_verify() (aliro_actuator.access_protocol.user_device.UserSession method), 32
 set_credential_ephemeral_key() (aliro_actuator.access_protocol.reader.ReaderSession method), 25
 set_credential_public_key() (aliro_actuator.access_protocol.reader.ReaderSession method), 25
 set_flag() (aliro_actuator.access_protocol.reader.ReaderSession method), 25
 set_logger() (aliro_actuator.Global class method), 41
 set_response_vendor_extension() (aliro_actuator.access_protocol.reader.ReaderSession method), 25
 set_select_info() (aliro_actuator.access_protocol.reader.ReaderSession method), 25
 set_shared_key() (aliro_actuator.access_protocol.reader.ReaderSession method), 26
 set_shared_key() (aliro_actuator.access_protocol.user_device.UserSession method), 32
 SET_TAG (aliro_actuator.access_protocol.defines.Exchange attribute), 15
 sign() (aliro_actuator.trust_framework.access_credential.AccessCredential method), 37
 sign() (aliro_actuator.trust_framework.key.KeyPair method), 39
 sign() (aliro_actuator.trust_framework.key.PrivateKey method), 39
 SIGNALING_BITMAP_LEN (aliro_actuator.access_protocol.defines.Auth1 attribute), 15
 SIGNALING_BITMAP_TAG (aliro_actuator.access_protocol.defines.Auth1 attribute), 15
 Socket (class in aliro_actuator.transport_protocol.socket), 36
 SOCKET_BLE (aliro_actuator.access_protocol.defines.TransportProtocol attribute), 16
 SOCKET_NFC (aliro_actuator.access_protocol.defines.TransportProtocol attribute), 16
 STANDARD (aliro_actuator.access_protocol.apdu.Transaction attribute), 13
 start() (aliro_actuator.access_protocol.user_device.MailboxSession method), 28
 start_new_session() (aliro_actuator.access_protocol.reader.ReaderSession method), 31
 START_OF_DATA_TAG (aliro_actuator.access_protocol.mailbox.Mailbox attribute), 20
 state_valid() (aliro_actuator.access_protocol.user_device.UserSession method), 32
 StatusBytes (class in aliro_actuator.access_protocol.apdu), 12
 step_up_aid_select_required() (aliro_actuator.access_protocol.reader.ReaderSession method), 26
 step_up_transaction() (aliro_actuator.access_protocol.reader.ReaderSession method), 25
 stop() (aliro_actuator.access_protocol.user_device.MailboxSession method), 28
 store() (aliro_actuator.access_document.Document method), 8
 SUCCESS (aliro_actuator.access_protocol.apdu.StatusBytes attribute), 13
 support_extended_length_apdu (aliro_actuator.access_protocol.Device property), 32
 sw1 (aliro_actuator.access_protocol.apdu.Response property), 12
 sw2 (aliro_actuator.access_protocol.apdu.Response property), 12
 technology (aliro_actuator.hw_driver.pn7160_driver.api.nfc_tag_info_t attribute), 33
 TECHNOLOGY_MASK (class in aliro_actuator.hw_driver.pn7160_driver.api), 33
 TLV (class in aliro_actuator.access_protocol.tlv), 26
 TlvError, 27
 to_bytes() (aliro_actuator.access_protocol.apdu.Message method), 11
 to_bytes() (aliro_actuator.access_protocol.tlv.TLV method), 27
 to_data() (aliro_actuator.access_protocol.tlv.TLV method), 27
 Transaction (class in aliro_actuator.access_protocol.apdu), 13
 TRANSACTION_CODE_LEN (aliro_actuator.access_protocol.defines.Auth0 attribute), 14
 TRANSACTION_CODE_TAG (aliro_actuator.access_protocol.defines.Auth0 attribute), 14
 TRANSACTION_ID_LEN (aliro_actuator.access_protocol.defines.Auth0 attribute), 14

TRANSACTION_ID_TAG (aliro_actuator.access_protocol.defines.TransactionCode attribute), 14

transaction_identifier (aliro_actuator.access_protocol.reader.ReaderSession property), 26

TRANSACTION_IDENTIFIER_TAG (aliro_actuator.access_protocol.defines.ReaderAuth attribute), 16

TRANSACTION_IDENTIFIER_TAG (aliro_actuator.access_protocol.defines.UserDeviceAuth attribute), 17

transaction_initiation() (aliro_actuator.access_protocol.reader.Reader method), 25

transaction_initiation() (aliro_actuator.access_protocol.user_device.UserDevice method), 31

TransactionCode (class in aliro_actuator.access_protocol.apdu), 13

TransportProtocol (class in aliro_actuator.access_protocol.defines), 16

TransportProtocolBase (class in aliro_actuator.transport_protocol), 36

TransportProtocolError, 35

TrustFrameworkError, 38

TYPE_LEN (aliro_actuator.access_protocol.defines.Select attribute), 16

TYPE_TAG (aliro_actuator.access_protocol.defines.Select attribute), 16

U

uid (aliro_actuator.hw_driver.pn7160_driver.api.nfc_tag_info_t attribute), 33

uid_length (aliro_actuator.hw_driver.pn7160_driver.api.nfc_tag_info_t attribute), 34

UnexpectedCommandError, 20

UnexpectedResponseError, 20

UPDATE_DOC_TAG (aliro_actuator.access_protocol.defines.Exchange attribute), 15

update_state() (aliro_actuator.access_protocol.user_device.UserSession method), 32

URSK_TAG (aliro_actuator.access_protocol.defines.Exchange attribute), 15

USAGE (aliro_actuator.access_protocol.defines.ReaderAuth attribute), 16

USAGE (aliro_actuator.access_protocol.defines.UserDeviceAuth attribute), 17

USAGE_TAG (aliro_actuator.access_protocol.defines.ReaderAuth attribute), 16

USAGE_TAG (aliro_actuator.access_protocol.defines.UserDeviceAuth attribute), 17

USER (aliro_actuator.access_protocol.encryption.DeviceType attribute), 17

USER_DEVICE (aliro_actuator.access_protocol.apdu.TransactionCode attribute), 13

USER_DEVICE_SECURE_ACTION (aliro_actuator.access_protocol.apdu.TransactionCode attribute), 13

USER_DEVICE_SIG_LEN (aliro_actuator.access_protocol.defines.Auth1 attribute), 15

USER_DEVICE_SIG_TAG (aliro_actuator.access_protocol.defines.Auth1 attribute), 15

UserDevice (class in aliro_actuator.access_protocol.user_device), 28

UserDeviceAuth (class in aliro_actuator.access_protocol.defines), 16

UserSession (class in aliro_actuator.access_protocol.user_device), 31

UserSessionState (class in aliro_actuator.access_protocol.user_device), 32

V

validate_credential() (aliro_actuator.access_document.access_credential.AccessDocument method), 7

VENDOR_SPECIFIC_MAX_LEN (aliro_actuator.access_protocol.defines.Auth0 attribute), 14

VENDOR_SPECIFIC_TAG (aliro_actuator.access_protocol.defines.Auth0 attribute), 14

VENDOR_SPECIFIC_TAG (aliro_actuator.access_protocol.defines.Select attribute), 16

VerificationError, 18

verify() (aliro_actuator.trust_framework.certificate.Certificate method), 37

verify() (aliro_actuator.trust_framework.key.KeyPair method), 39

verify() (aliro_actuator.trust_framework.key.PublicKey method), 39

VersionError, 20

W

wait_for_command() (aliro_actuator.access_protocol.user_device.UserDevice method), 31

wait_for_connection() (aliro_actuator.transport_protocol.ble_uwb.BLEUWB method), 35

wait_for_connection() (aliro_actuator.transport_protocol.nfc.NFC method), 35

`wait_for_connection()`
 (*aliro_actuator.transport_protocol.socket.Socket*
 method), 36

`wait_for_connection()`
 (*aliro_actuator.transport_protocol.TransportProtocolBase*
 method), 36

`wait_for_reader()` (*aliro_actuator.hw_driver.pn7160_driver.Driver*
 method), 34

`wait_for_tag()` (*aliro_actuator.hw_driver.pn7160_driver.Driver*
 method), 34

`WRITE` (*aliro_actuator.access_protocol.user_device.MailboxSession.Command*
 attribute), 28

`write()` (*aliro_actuator.access_protocol.mailbox.Mailbox*
 method), 21

`WRITE_TAG` (*aliro_actuator.access_protocol.defines.Exchange*
 attribute), 15

`WRONG_LENGTH_IN_LC` (*aliro_actuator.access_protocol.apdu.StatusBytes*
 attribute), 13